

**Dipartimento di Informatica
Università di Pisa**

Laboratorio 2 - Parte II

Dispensa

Patrizio Dazzi

a.a. 2024-25

© Patrizio Dazzi, 2025.

© ⓘ ⓘ Questo documento è rilasciato sotto licenza **Creative Commons Attribuzione-Condividi allo stesso modo 4.0 Internazionale (CC BY-SA 4.0)**. Puoi trovare il testo completo della licenza all'indirizzo:
<https://creativecommons.org/licenses/by-sa/4.0/deed.it>

Colophon

Questo documento è stato impaginato con l'aiuto di **KOMA-Script** e **L^AT_EX** utilizzando la classe **kaobook**.
Il codice sorgente di questo libro è disponibile all'indirizzo:
<https://github.com/fmarotta/kaobook>

Indice

Indice	iii
1 Introduzione	1
1.1 Scopo della dispensa	1
1.2 Struttura	1
 I COMPILAZIONE E PRECOMPILAZIONE	 3
2 make e compilazione separata	5
2.1 Introduzione a make	5
2.1.1 Origini e Motivazioni	5
2.1.2 Struttura di un Makefile	5
2.1.3 Albero delle Dipendenze	6
2.2 Compilazione separata	7
2.2.1 Esempio di Makefile per Progetti Modulari	7
2.2.2 Strategie Avanzate	7
 3 Precompilazione C	 11
3.1 Introduzione al preprocessore	11
3.1.1 #include	11
3.1.2 #define	12
3.2 Direttive di precompilazione	12
3.2.1 Direttive di inclusione	12
3.2.2 Direttive di definizione	13
3.2.3 Direttive condizionali	13
3.2.4 Direttive di errore e avviso	14
3.2.5 Direttive per token e concatenazione	14
3.2.6 Direttive per il controllo di file	14
3.2.7 Direttive di inclusione multipla	15
3.2.8 Macro predefinite standard in C11	16
3.3 Regole generali	16
3.3.1 Devono iniziare con il carattere #	16
3.3.2 Finiscono con il termine della riga corrente a meno di backslash	16
3.3.3 In qualsiasi punto del programma	16
3.3.4 Commenti sulla stessa linea della direttiva	17
3.4 Macro semplici	17
3.5 Macro parametriche	18
3.5.1 Macro con espressioni multiple	21
3.5.2 Macro con istruzioni multiple	22
3.5.3 Attenzione agli spazi prima delle parentesi	24
3.5.4 Adozione di uno stile simil-funzionale	24
3.6 Macro annidate	24
3.6.1 Nessuna Ricorsione	24
3.6.2 Un esempio corretto	25

II	PROGRAMMAZIONE CONCORRENTE CON C11	27
4	Programmazione concorrente con C11: Ciclo di vita dei thread	29
4.1	Programmazione concorrente (in a nutshell)	29
4.1.1	Più tipi di Thread	31
4.1.2	Thread e memoria	33
4.2	Supporto ad applicazioni concorrenti in C11	33
4.3	Threading: elementi di base	34
4.3.1	La creazione: <code>thrd_create</code>	34
4.3.2	L'identificazione: <code>thrd_current</code>	35
4.3.3	L'unione, l'attesa: <code>thrd_join</code>	36
4.3.4	L'indipendenza: <code>thrd_detach</code>	37
4.3.5	La terminazione: <code>thrd_exit</code>	37
5	Programmazione concorrente con C11: Operazioni Atomiche e Lock	41
5.1	Premessa	41
5.2	Operazioni Atomiche	41
5.2.1	Definizione di Atomicità	41
5.2.2	Supporto in C11	42
5.2.3	Classi di Memoria Atomica	43
5.3	Meccanismi di Lock	44
5.3.1	Spinlock	44
6	Programmazione concorrente con C11: Mutex e Condition Variables	47
6.1	Mutex	47
6.1.1	Dichiarazione e Inizializzazione di un Mutex	47
6.1.2	Lock e Unlock su mutex	49
6.1.3	Deadlock e Gestione delle Risorse	49
6.2	Condition Variables	49
6.2.1	Condition Variables in C11	50
6.2.2	Spurious Wakeups	51
6.2.3	Esempio di Produttori-Consumatori con Condition Variables	52

1.1 Scopo della dispensa

1.1 Scopo della dispensa 1

1.2 Struttura 1

Questa dispensa ha l'obiettivo di fornire agli studenti una guida pratica e teorica per affrontare il corso di "Laboratorio 2". Essa copre una varietà di argomenti essenziali per sviluppatori, con un focus sulle tecniche di compilazione, il preprocessing in linguaggio C e la programmazione concorrente. L'intento è quello di approfondire i concetti fondamentali, offrendo esempi pratici e strategie utili per l'applicazione quotidiana nello sviluppo software.

Tra i temi principali trattati, vi sono:

- ▶ Introduzione e uso del comando make per automatizzare la compilazione.
- ▶ Tecniche di compilazione separata per progetti complessi.
- ▶ Utilizzo avanzato del preprocessore in C, incluse macro e direttive condizionali.
- ▶ Fondamenti di programmazione concorrente con il linguaggio C11, con particolare attenzione alla gestione dei thread e alla sincronizzazione.
- ▶ Chiamate di sistema
- ▶ Comunicazione inter-processo

1.2 Struttura

La dispensa è suddivisa in capitoli tematici, ciascuno dei quali affronta un aspetto specifico dello sviluppo software:

1. Make e compilazione separata: Introduce il comando make, le sue origini, e il suo utilizzo per gestire progetti software modulari, descrivendo la struttura di un Makefile e le regole implicite per diversi linguaggi di programmazione.
2. Precompilazione in linguaggio C: Approfondisce il ruolo del preprocessore, incluse le direttive più comuni (`#include`, `#define`) e le macro, fornendo esempi pratici e regole per evitare errori.
3. Programmazione concorrente con C11: Fornisce una panoramica sui concetti fondamentali di programmazione concorrente, spiegando il supporto offerto dallo standard C11 per la gestione dei thread, la mutua esclusione e le operazioni atomiche.
4. Chiamate di sistema: Esamina la struttura e le principali chiamate di sistema per la gestione delle risorse e dei processi, con esempi di codice.
5. Comunicazione tra processi (IPC): Introduce alcuni meccanismi per lo scambio di dati tra processi, come pipe, code di messaggi e memoria condivisa.

Ogni concetto è introdotto assieme a semplici esempi pratici per favorire l'apprendimento attivo e una comprensione più profonda degli argomenti. Le appendici offrono ulteriori dettagli tecnici e riferimenti per approfondimenti.

Parte I

COMPILAZIONE E PRECOMPILAZIONE

2.1 Introduzione a make

Il comando `make` rappresenta una pietra miliare nello sviluppo software su sistemi Unix e Unix-like. Nato negli anni '70, il suo scopo principale è automatizzare il processo di costruzione del software, gestendo in modo efficiente la compilazione di progetti complessi. Attraverso un file chiamato `Makefile`, `make` consente agli sviluppatori di descrivere le dipendenze tra i file e di definire le azioni necessarie per aggiornare i target, rendendo il processo ripetibile e meno soggetto a errori.

In questo capitolo discuteremo brevemente i principi di funzionamento di `make`, il concetto di compilazione separata e il modo in cui questi strumenti si integrano per semplificare lo sviluppo software. Analizzeremo inoltre esempi pratici e strategie avanzate per sfruttare al meglio questo strumento.

2.1.1 Origini e Motivazioni

Negli anni '70, la crescente complessità dei software rese evidente la necessità di uno strumento per gestire le dipendenze tra file sorgenti e automatizzare la loro compilazione. Prima dell'introduzione di `make`, ogni modifica al codice richiedeva l'intervento manuale per ricompilare i file interessati e rigenerare l'eseguibile. Questo processo era inefficiente e soggetto a frequenti errori umani.

`make` fu progettato per risolvere questo problema: è uno strumento che segue il principio "compila solo ciò che è necessario". Questo significa che solo i file modificati e quelli che dipendono da essi vengono ricompilati, riducendo drasticamente i tempi di compilazione.

2.1.2 Struttura di un Makefile

Un `Makefile` è il cuore del sistema `make`. Esso descrive:

- Obiettivo: il risultato da generare (ad esempio, un file oggetto o un eseguibile).
- Prerequisiti: i file da cui l'obiettivo dipende.
- comando: i comandi necessari per generare o aggiornare l'obiettivo (si noti il tab obbligatorio prima di ogni comando).

Sintassi di base

Una regola in un `Makefile` segue la struttura seguente:

2.1	Introduzione a make . . .	5
2.1.1	Origini e Motivazioni . .	5
2.1.2	Struttura di un Makefile	5
2.1.3	Albero delle Dipendenze	6
2.2	Compilazione separata .	7
2.2.1	Esempio di Makefile per Progetti Modulari	7
2.2.2	Strategie Avanzate	7

Listato 2.1: Sintassi di base di una regola

```
# commento

obiettivo: prerequisiti
    ──▶ comando
    ──▶ comando
    ──▶ ...
```

Vediamo quindi un primo esempio di regola:

Listato 2.2: Sintassi di base di una regola

```
main.o: main.c utils.h
    ──▶ gcc -Wall -c main.c ◀
```

In questo caso `main.o` dipende da `main.c` e `utils.h`. Se uno di questi file viene modificato, il comando `gcc -Wall -c main.c` viene eseguito per rigenerare `main.o`.

La struttura del Makefile presenta alcune peculiarità che richiedono attenzione:

- ▶ Uso dei tab: i comandi elencati sotto un target devono iniziare con un carattere di tabulazione (rappresentato da  nel frammento di codice descritto sopra) e non con spazi. L'uso errato degli spazi genera errori difficili da diagnosticare.
- ▶ Linee vuote: è necessario lasciare almeno una linea vuota tra una regola e l'altra per separarle correttamente.
- ▶ Newline finale: il Makefile deve terminare con una nuova riga vuota (rappresnetato dal simbolo  nel frammento di codice); in caso contrario, alcune versioni di make potrebbero segnalare errori.

2.1.3 Albero delle Dipendenze

Uno degli aspetti più potenti di `make` è la capacità di costruire un albero delle dipendenze. Questo albero rappresenta le relazioni tra i file in un progetto e guida il processo di compilazione.

1. Costruzione dell'albero: `make` parte dalla prima regola del Makefile e costruisce l'albero seguendo le dipendenze dichiarate.
2. Visita bottom-up: durante l'esecuzione, `make` visita l'albero dal basso verso l'alto. Ogni nodo viene aggiornato solo se necessario, ossia se una delle sue dipendenze è stata modificata o non esiste.

```
main
|
|-- main.o
|   |-- main.c
|   |-- utils.h
|
|-- utils.o
    |-- utils.c
    |-- utils.h
```

Se `utils.h` viene modificato, sia `main.o` che `utils.o` saranno rigenerati, seguiti dal linking per produrre `main`.

2.2 Compilazione separata

Nel campo dei progetti software, specialmente quando si tratta di sistemi complessi, è comune adottare la pratica della *compilazione separata*. Questo approccio prevede che il codice sorgente sia suddiviso in moduli piuttosto che essere compilato in un'unica volta. Ogni modulo è trasformato in un file oggetto singolo e autonomo (con estensione `.o`), il quale verrà unito agli altri durante il processo di linking finale. I vantaggi risultanti sono molteplici:

- ▶ Efficienza: solo i moduli modificati vengono ricompilati.
- ▶ Manutenibilità: facilita l'organizzazione del codice e il riutilizzo.
- ▶ Modularità: consente di suddividere il lavoro tra diversi sviluppatori.

2.2.1 Esempio di Makefile per Progetti Modulari

Supponiamo di avere un progetto con i seguenti file:

- ▶ `main.c`: contiene il punto di ingresso del programma.
- ▶ `utils.c` e `utils.h`: forniscono funzioni di utilità.

Ecco un Makefile che implementa la compilazione separata:

```
CC = gcc
CFLAGS = -Wall -pedantic
OBJECTS = main.o

all: main

main: $(OBJECTS)
    $(CC) $(CFLAGS) -o main $(OBJECTS)

main.o: main.c
    $(CC) $(CFLAGS) -c main.c

clean:
    rm -f $(OBJECTS) main
```

Listato 2.3: Esempio di makefile

Esaminiamo la questione con maggiore attenzione:

1. Variabili: `CC` definisce il compilatore, `CFLAGS` contiene le opzioni di compilazione.
2. Obiettivo principale (`all`): genera l'eseguibile.
3. Regole per i file oggetto: ciascun file sorgente è compilato in un file oggetto indipendente.
4. Obiettivo `clean`: rimuove i file generati, utile per una ricompilazione completa.

2.2.2 Strategie Avanzate

Uso di Regole Implicite

`make` supporta regole implicite, che riducono la necessità di specificare comandi ripetitivi. Queste regole predefinite gestiscono automaticamente

la compilazione di file oggetto a partire dai rispettivi file sorgenti. Ad esempio, `make` sa che per generare `main.o` da `main.c`, deve eseguire un comando simile a:

```
gcc -c main.c -o main.o
```

Linguaggi Supportati

Le regole implicite di `make` coprono diversi linguaggi di programmazione, tra cui:

- ▶ C: file sorgenti `.c` compilati in `.o`.
- ▶ C++: file sorgenti `.cpp` o `.cc` compilati in `.o`.
- ▶ Fortran: file `.f` o `.f90` compilati in `.o`.
- ▶ Assembler: file `.s` o `.asm` compilati in `.o`.
- ▶ Lex/Yacc: file `.l` e `.y` trasformati in `.c` tramite `lex` e `yacc`.

Per ciascuno di questi linguaggi, `make` utilizza regole predefinite basate su variabili standard come `CC` per il compilatore C o `CXX` per il compilatore C++. Queste regole possono essere personalizzate modificando tali variabili o aggiungendo nuove regole nel `Makefile`.

Questo significa che è possibile realizzare una versione semplificata del `Makefile` nella quale non sono necessarie regole esplicite per `main.o` o `utils.o`, poiché `make` applicherà automaticamente queste regole implicite quando i file sorgenti corrispondenti esistono e sono specificati come dipendenze. Il `Makefile` semplificato diventa quindi più conciso, affidandosi al comportamento predefinito di `make`. Ad esempio, il `Makefile` precedente può essere semplificato:

Listato 2.4: Esempio di `makefile` semplificato

```
CC = gcc
CFLAGS = -Wall -pedantic
OBJECTS = main.o

all: main

main: $(OBJECTS)
    $(CC) $(CFLAGS) -o $@ $^

clean:
    rm -f $(OBJECTS) main
```

Qui, `$@` rappresenta il target corrente e `$^` le sue dipendenze.

Regole Generiche o Pattern Rule

Oltre alle regole implicite, `make` permette di definire regole generiche, dette *pattern rule*, che applicano lo stesso comando a un gruppo di file con caratteristiche comuni. Ad esempio:

```
%.o : %.c
    gcc -c -o $@ $<
```

- ▶ `%.o`: rappresenta qualsiasi file con estensione `.o`.
- ▶ `%.c`: rappresenta qualsiasi file con estensione `.c` con lo stesso nome base del `.o` corrispondente.
- ▶ `$@`: è una variabile automatica che rappresenta il target (il file `.o` da generare).

- `$<`: è una variabile automatica che rappresenta la prima dipendenza (il file sorgente `.c`).

Con questa regola, `make` compilerà automaticamente file come `main.o` da `main.c` o `utils.o` da `utils.c` senza bisogno di regole esplicite per ciascun file.

Esempio Immaginiamo di avere due file da compilare, `main.c` e `utils.c`. Se nel Makefile è presente la regola

```
%o : %.c
    gcc -c -o $@ $<
```

a fronte dell'esecuzione di `make main.o` il Makefile espande la regola generica in: `gcc -c -o main.o main.c`. In altre parole, `$@` corrisponde a `main.o` mentre `$<` corrisponde a `main.c` (la prima dipendenza). Al contrario, se anziché utilizzare `$<` si utilizzasse `^`, questo coinvolgerebbe tutte le dipendenze. Per intenderci, una regola che fosse scritta in questo modo:

```
program: main.o utils.o
    gcc -o $@ $^
```

Eseguendo `make program`, il comando risultante sarebbe `gcc -o program main.o utils.o`.

Phony Targets

I target come `clean` non corrispondono a file reali. Per evitare conflitti, si dichiara esplicitamente:

```
.PHONY: clean
clean:
    rm -f $(OBJECTS) main
```

Variabili

È inoltre possibile giocare con le variabili per definire Makefile più ricchi ed elaborati. Si possono definire delle variabili specifiche per diversi obiettivi (target), aggiungendo a variabili già definite ulteriori parametri. A titolo di esempio si consideri un Makefile che sia in grado di gestire sia la versione di release che di debug di un software. La prima deve, anche a costo di un maggiore tempo di compilazione, procedere ad applicare ottimizzazioni al codice, l'altra deve includere i simboli utili al processo di debug.

```
CC = gcc
CFLAGS = -Wall -pedantic
DEBUG_FLAGS = -g $(CFLAGS)
RELEASE_FLAGS = -O3 $(CFLAGS)
OBJECTS = main.o

all: debug release

main: $(OBJECTS)

%.o: %.c
    $(CC) -c $(CFLAGS) $< -o $@
```

Listato 2.5: Esempio di Makefile avanzato

```
debug: main
    $(CC) $(DEBUG_FLAGS) main.o -o debug

release: main
    $(CC) $(RELEASE_FLAGS) main.o -o release
    zip release.zip release

clean:
    rm -f $(OBJECTS) main debug release release.zip

.PHONY: debug release clean
```

Inoltre, per la versione di release può essere utile generare un file zip contenente l'intero progetto.

Conclusioni

Il sistema make e il concetto di compilazione separata sono strumenti potenti e fondamentali nello sviluppo software. Comprendere come utilizzare al meglio make consente di ottimizzare il processo di build, migliorando efficienza e manutenibilità.

Eseguendo comandi relativamente semplici si ha la possibilità di condurre processi di compilazione anche piuttosto articolati. Esistono alcune buone prassi che è bene seguire in fase di definizione del Makefile. In particolare, risulta utile riservare un target denominato `run` per eseguire il programma, uno denominato `clean` per rimuovere in automatico gli artefatti generati in fase di compilazione. Quando il Makefile si riferisce a più sottoprogetti è auspicabile anche fornire un target `all` che determini la compilazione di tutti i moduli.

Il preprocessore è un programma che viene attivato dal compilatore nella fase precedente alla compilazione, detta di precompilazione. Esso opera antecedentemente all'entrata in funzione effettiva del compilatore. La sua principale funzione consiste nel trasformare il codice sorgente prima della sua trasmissione al compilatore, permettendo così la successiva produzione del codice eseguibile.

Il preprocessore è un componente estremamente adattabile che conferisce un carattere distintivo e una notevole potenza al linguaggio di programmazione C. Tuttavia, il suo impiego può diventare detrimentalmente in assenza di una comprensione approfondita del suo funzionamento. Questo capitolo è dedicato specificamente a chiarire il contributo del preprocessore all'interno del ciclo di compilazione di un programma sviluppato utilizzando il linguaggio C.

3.1 Introduzione al preprocessore

Il funzionamento del preprocessore è governato dalle Direttive di Precompilazione, che consistono in specifiche istruzioni a cui il preprocessore risponde modificando il codice sorgente. Una volta che queste modifiche vengono apportate, il preprocessore genera il risultato finale dell'operazione. Questo risultato viene quindi fornito come input al compilatore, il cui compito è convertirlo nel codice oggetto, essenziale per la fase esecutiva dei programmi. Il ruolo del processo di precompilazione è schematizzato nella Fig. 3.1 Tutte le direttive di precompilazione iniziano con il simbolo “#” e concludono alla fine della linea in cui sono scritte. In passato, avete già incontrato due tipi di direttive: `#include` e `#define`.

3.1.1 `#include`

La direttiva `#include` consente di incorporare il contenuto di un file sorgente in un altro, offrendo un modo efficiente di suddividere e organizzare il codice sorgente in diversi file. Fino ad ora, avete impiegato questa direttiva principalmente per includere file come `stdio.h`, che comprendono le dichiarazioni delle funzioni appartenenti alla libreria standard.

Il preprocessore implementa una direttiva `#include` eseguendo le seguenti operazioni:

- Apre il file specificato dalla direttiva `#include` all'interno delle directory di sistema e ne legge il contenuto (lanciando un errore nel caso in cui il file non venisse trovato);
- Sostituisce ogni occorrenza della direttiva `#include` con il contenuto testuale del file.

3.1	Introduzione al preprocessore	11
3.1.1	<code>#include</code>	11
3.1.2	<code>#define</code>	12
3.2	Direttive di precompilazione	12
3.2.1	Direttive di inclusione	12
3.2.2	Direttive di definizione	13
3.2.3	Direttive condizionali	13
3.2.4	Direttive di errore e avviso	14
3.2.5	Direttive per token e concatenazione	14
3.2.6	Direttive per il controllo di file	14
3.2.7	Direttive di inclusione multipla	15
3.2.8	Macro predefinite standard in C11	16
3.3	Regole generali	16
3.3.1	Devono iniziare con il carattere #	16
3.3.2	Finiscono con il termine della riga corrente a meno di backslash	16
3.3.3	In qualsiasi punto del programma	16
3.3.4	Commenti sulla stessa linea della direttiva	17
3.4	Macro semplici	17
3.5	Macro parametriche	18
3.5.1	Macro con espressioni multiple	21
3.5.2	Macro con istruzioni multiple	22
3.5.3	Attenzione agli spazi prima delle parentesi	24
3.5.4	Adozione di uno stile simil-funzionale	24
3.6	Macro annidate	24
3.6.1	Nessuna Ricorsione	24
3.6.2	Un esempio corretto	25

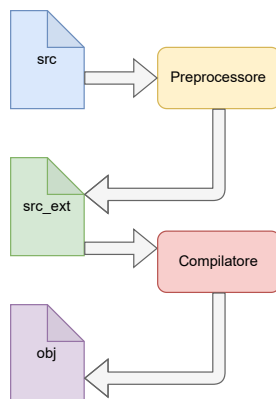


Figura 3.1: Preprocessore

Preprocessore

Il preprocessore è un software che riceve in input un file sorgente scritto in linguaggio C e ne apporta modifiche prima della consegna al compilatore. Le trasformazioni effettuate dal preprocessore sono guidate dalle direttive precompilative. Quando il preprocessore rileva una direttiva, la esamina e la applica. Una volta eseguita una direttiva, il preprocessore altera il file sorgente rimuovendo la direttiva stessa.

Sette tipologie di direttive

- inclusione
- definizione
- condizionali
- errore e avviso
- token e concatenazione
- controllo file
- inclusione multipla

3.1.2 #define

La direttiva `#define` consente di creare ciò che viene chiamato una *macro*. Una macro è un identificativo simbolico che assegniamo a un valore o a un'espressione. Questo nome simbolico può poi essere impiegato in qualsiasi parte del codice sorgente come sostituto del valore o dell'espressione che rimpiazza.

Una semplice macro definita con la direttiva `#define` è la seguente:

```
#define PI 3.14
```

Con questa macro abbiamo espresso simbolicamente la costante `PI`, assegnandogli un valore letterale, in questo caso l'espressione numerica `3.14`. Il preprocessore prende in input un file sorgente ed effettua due operazioni:

- Memorizza, il nome della macro insieme al valore o all'espressione che rappresenta.
- Sostituisce, ogni volta che la incontra la macro con il valore o l'espressione che rappresenta.

In entrambi i casi, il file sorgente viene modificato a livello testuale. Il ruolo del preprocessore è generalmente invisibile. Durante l'uso quotidiano, non è necessario richiamare il preprocessore in modo esplicito; basta semplicemente richiamare il compilatore, il quale automaticamente provvederà a chiamare il preprocessore passando il file sorgente in questione. Per esempio, quando si esegue il compilatore `gcc` su un file sorgente, esso attiva automaticamente il proprio preprocessore, denominato `cpp` in ambiente Linux, dove indica C PreProcessor, mentre su altri sistemi può coincidere con il compilatore stesso.

3.2 Direttive di precompilazione

Le direttive di precompilazione (o per preprocessore) in C11 (e negli standard precedenti) sono strumenti forniti dal preprocessore del linguaggio C per gestire il codice prima della compilazione. Queste direttive non sono cambiate significativamente nello standard C11 rispetto ai precedenti. Ecco un elenco delle direttive di precompilazione standard in C11:

3.2.1 Direttive di inclusione

Si tratta della `#include` già menzionata precedentemente. Include il contenuto di un file di intestazione (header file).

```
#include <file> // Cerca il file nelle directory di sistema.
#include "file" // Cerca prima nella directory corrente, poi nelle
                // directory di sistema.
```


3.2.2 Direttive di definizione

La direttiva `#define` definisce un macro. Può essere utilizzata per costanti o macro con parametri. Al contrario, la direttiva `#undef` annulla (*undefine*) una macro precedentemente definita.

```
#define PI 3.14
#define QUADRATO(x) ((x) * (x))
#undef PI
```

3.2.3 Direttive condizionali

Queste direttive permettono la compilazione condizionale del codice.

- ▶ `#if` Inizia una struttura condizionale, valutando un'espressione costante.
- ▶ `#ifdef` Compila il codice solo se la macro è definita.
- ▶ `#ifndef` Compila il codice solo se la macro non è definita.
- ▶ `#elif` Aggiunge una condizione alternativa in una struttura `#if`.
- ▶ `#else` Aggiunge un'alternativa finale in una struttura `#if`
- ▶ `#endif` Termina una struttura condizionale iniziata con `#if`, `#ifdef` o `#ifndef`.

Il Listato 3.1 include alcuni esempi di base che illustrano l'utilità dell'impiego delle direttive condizionali in vari contesti.

```
1 #include <stdio.h>
2
3 #define DEBUG
4 #define VERSION 2
5
6 int main() {
7     #ifdef DEBUG
8         printf("Debug mode\n");
9     #endif
10
11     #ifndef RELEASE
12         printf("Not a Release\n");
13     #endif
14
15     #if VERSION == 1
16         printf("Version 1\n");
17     #elif VERSION == 2
18         printf("Version 2\n");
19     #else
20         #error "Unsupported.\n"
21     #endif
22
23     #ifdef DEBUG
24         printf("Performing additional debug checks...\n");
25     #endif
26
27     #line 100 "custom_file.c"
28
29     printf("Code from %s at line %d.\n", __FILE__, __LINE__);
30     return 0;
31 }
```

Listato 3.1: Esempio di programma con direttive condizionali

3.2.4 Direttive di errore e avviso

`#error` Genera un messaggio di errore e interrompe la compilazione.
`#warning` (non standard, ma supportato da molti compilatori) Genera un messaggio di avviso durante la compilazione (senza interromperla).

```
#error "Unsupported platform"
#warning "Deprecated feature"
```

3.2.5 Direttive per token e concatenazione

`#` (stringizzazione): Trasforma un argomento in una stringa letterale.
`##` (concatenazione): Concatena due token in un unico token.

```
#define TO_STRING(x) #x
printf(TO_STRING(123)); // Output: "123"

#define CONCAT(a, b) a##b
int CONCAT(var, 1) = 10; // Definisce int var1 = 10;
```

Nota: andando a passare come argomento alla macro di stringizzazione un'espressione, come ad esempio `2*3`, il risultato sarà effettivamente la stringa `"2*3"`, ovvero l'espressione specificata come argomento non è valutata prima di essere trasformata in stringa. A titolo di esempio si consideri il Listato 3.2 riportato di seguito.

Listato 3.2: Stringizzazione

```
#include <stdio.h>
#define TO_STRING(x) #x

int main(){
    printf("%s\n", TO_STRING(2*3));
    return 0;
}
```

Il risultato dell'esecuzione sarà:

```
2*3
```

3.2.6 Direttive per il controllo di file

`#line`: Cambia il numero di riga e il nome del file per i messaggi di errore e avviso.

Listato 3.3: Direttive per il controllo di file

```
#line 100 "newfile.c" // Questo risulta utile per il debug.
```

`#pragma`: Fornisce istruzioni al compilatore, ma il comportamento è specifico per il compilatore. Alcuni esempi comuni:

- `#pragma once`: Evita inclusioni multiple di un file header (equivalente a guardie di inclusione).
- `#pragma GCC`, `#pragma warning`, ecc.: Usate per configurazioni specifiche del compilatore.

3.2.7 Direttive di inclusione multipla

Le guardie di inclusione evitano la pluri-inclusione degli header:

```
#ifndef HEADER_FILE
#define HEADER_FILE
// Codice del file header
#endif
```

Le guardie di inclusione sono essenziali nei file di intestazione (.h) per evitare inclusioni multiple dello stesso file, che possono portare a errori di ridefinizione durante la compilazione. Questo problema si verifica quando un file .h viene incluso più volte, direttamente o indirettamente, in più unità di traduzione (.c).

Quando non sono necessarie?

Le guardie di inclusione non sono necessarie se un file .h contiene solo dichiarazioni (senza definizioni), il codice si compila correttamente anche senza guardie. Ad esempio, questo codice:

```
int funzione(int arg);
```

può essere incluso più volte senza causare errori. Tuttavia, è buona pratica proteggerlo con guardie di inclusione per evitare problemi futuri.

Anche la definizione di macro non richiede guardie di inclusione, perché vengono espanse dal preprocessore e non creano simboli nel codice oggetto. Bisogna evitare ridefinizioni incoerenti in file diversi.

Quando sono necessarie?

Le guardie di inclusione sono necessarie se l'header file che si vuole includere contiene una definizione di variabile globale come, ad esempio:

```
int x = 42; // Definizione della variabile (non solo
           // dichiarazione)
```

allora ogni file .c che include prova.h genererà una copia separata della variabile. In fase di linking, il compilatore segnerà un errore di "multiple definition". Per evitare questo, la soluzione possibile è dichiarare la variabile extern negli header file e definirla in un solo file .c.

Le guardie di inclusione sono necessarie anche quando le funzioni sono definite direttamente nell'header file. Infatti, in quel caso verrà inclusa e ridefinita in ogni .c che include prova.h, causando errori di linking. La soluzione corretta è dichiarare la funzione negli header file, in questo modo:

```
int funzione(int arg);
```

e definirla soltanto in un file .c, come ad esempio il seguente:

```
int funzione(int arg){
    return arg+1;
}
```

Differenze principali con C23

C23 apporta alcune innovazioni, tra cui il miglioramento delle direttive di precompilazione e delle macro, pur mantenendo sostanzialmente invariati i principi fondamentali. Un esempio di queste innovazioni è l'inclusione di macro dedicate alla gestione delle funzionalità opzionali.

3.2.8 Macro predefinite standard in C11

Alcune macro sono definite automaticamente dal compilatore:

- ▶ `__DATE__`: Data di compilazione (es. "Jan 10 2025").
- ▶ `__TIME__`: Ora di compilazione (es. "14:23:45").
- ▶ `__FILE__`: Nome del file sorgente corrente.
- ▶ `__LINE__`: Numero di riga corrente nel file.
- ▶ `__STDC__`: Definito se il compilatore è conforme allo standard C.
- ▶ `__STDC_VERSION__`: Versione dello standard C. In C11 vale 201112L.

3.3 Regole generali

Le direttive di precompilazione, pur utilizzate per molteplici obiettivi, devono aderire a un insieme uniforme di standard che assicuri la loro corretta applicazione e interoperabilità nei contesti specificati.

Oltre agli esempi precedentemente illustrati, cerchiamo di delineare con maggiore precisione le norme che regolano la definizione delle direttive:

3.3.1 Devono iniziare con il carattere #

Le direttive devono iniziare con il simbolo `#`, che deve essere il primo carattere significativo della linea dove compaiono. Tuttavia, prima del simbolo `#` possono esserci spazi o tabulazioni, consentendo l'indentazione delle direttive se necessario.

È importante sottolineare che non è obbligatorio che la direttiva sia posizionata all'inizio della linea.

3.3.2 Finiscono con il termine della riga corrente a meno di backslash

La terminazione di ciascuna direttiva avviene alla fine della sua riga. Ciò significa che non è consentito dividere una direttiva su più righe, salvo specifiche modifiche.

Se una direttiva è troppo lunga per essere contenuta in una sola riga, si deve utilizzare più direttive, suddividendole con il carattere `\` (backslash) alla fine di ogni riga, tranne che nella riga finale.

3.3.3 In qualsiasi punto del programma

Le direttive di precompilazione possono essere inserite in qualunque posizione all'interno del programma. Non devono obbligatoriamente essere poste all'inizio del file sorgente.

Tuttavia, è considerata una buona abitudine raggrupparle all'inizio per una migliore leggibilità e organizzazione del codice.

3.3.4 Commenti sulla stessa linea della direttiva

Le direttive di precompilazione possono includere commenti. Questi commenti possono essere presenti direttamente sulla stessa riga della direttiva oppure su righe che la seguono.

Gli elementi costitutivi di una direttiva possono essere distinti da una quantità variabile di spazi e tabulazioni.

3.4 Macro semplici

Un insieme di istruzioni, funzioni o espressioni può essere raggruppato e identificato mediante un nome simbolico, creando una macro. Questo nome simbolico serve per richiamare il blocco di codice nei punti desiderati. Anche se una macro può sembrare simile a una funzione, il funzionamento è distintamente diverso.

Mentre le funzioni vengono eseguite durante l'esecuzione del programma, le macro sono espanse o sostituite dal preprocessore prima della compilazione effettiva del programma. Questo procedimento è conosciuto come sostituzione testuale, e nel linguaggio tecnico si dice che la macro si espande. Analizziamo questo processo in modo più approfondito.

La sintassi per definire una macro semplice è la seguente:

```
#define nome_macro testo_macro
```

Dove:

- nome_macro è il nome simbolico della macro.
- testo_macro è il testo che verrà sostituito al posto del nome.

Un aspetto cruciale da sottolineare è l'impiego del termine *testo da sostituire*. In realtà, il testo o il corpo della macro può consistere in qualsiasi sequenza di caratteri.

Questa osservazione suggerisce che la macro possieda la capacità di operare sostituzioni su una vasta gamma di componenti, dimostrando una notevole versatilità che va oltre il semplice ambito di specifiche istruzioni o espressioni. In altre parole, la macro è progettata per funzionare in modo estensivo, adattandosi a molteplici scenari e applicazioni con efficacia.

L'esempio tipico di utilizzo è la definizione di costanti numeriche oppure costanti di conversione come il numero di secondi in un minuto o ore in un giorno. Alcuni esempi sono riportati nel Listato seguente:

```
#define PI 3.14
#define E 2.72
#define SECONDI_PER_MINUTO 60
#define ORE_IN_UN_GIORNO 24
```

In tal caso, ogni qualvolta avremo la necessità di impiegare una di queste costanti all'interno del nostro programma, avremo la possibilità di fare riferimento al nome simbolico della macro corrispondente. Per esempio:

```
float cerchio(float raggio) {
    return PI * raggio * raggio;
}
```

Una volta espansa la macro, diventa:

```
float cerchio(float raggio) {
    return 3.14 * raggio * raggio;
}
```

Poiché all'interno di una macro può essere incluso qualsiasi testo, è possibile sfruttare le macro per realizzare sostituzioni più sofisticate. Riprendendo l'esempio precedente della funzione, immaginiamo di avere frequentemente la necessità di calcolare il valore dell'area di un cerchio con raggio 2 nel nostro programma. Anziché chiamare ripetutamente la funzione `area_cerchio` passando 2 come parametro, possiamo creare una macro che sostituisca direttamente l'uso della funzione. Ad esempio:

```
#define AREA_CERCHIO_2 cerchio(2.0)
```

Nel nostro programma, è possibile utilizzare la denominazione simbolica associata alla macro `AREA_CERCHIO_2`. Sarà compito del preprocessore effettuare la sostituzione del nome simbolico con il suo corrispondente contenuto. Per esempio:

```
printf("area di un cerchio di raggio 2: %f\n", AREA_CERCHIO_2);
```

diventa:

```
printf("area di un cerchio di raggio 2: %f\n", area_cerchio(2.0));
```

Da notare che la sostituzione avviene a livello di testo. Il preprocessore non sostituisce la macro con il risultato dell'invocazione della funzione. Quindi la funzione `area_cerchio` viene invocata ogni volta che usiamo il nome simbolico della macro.

Nota Bene: Durante la fase di sostituzione, il preprocessore non verifica l'esattezza e la validità dei simboli e del testo che sono destinati a essere sostituiti all'interno della macro. Questa omissione può essere fonte di complicazioni nella fase di compilazione del codice. Consideriamo, ad esempio, la funzione `cerchio`: qualora tale funzione non fosse già stata definita in anticipo, la macro procederebbe comunque con la sostituzione, determinando inevitabilmente un errore di compilazione.

3.5 Macro parametriche

Nella precedente lezione, abbiamo esplorato la sintassi utilizzata per la definizione di macro nel linguaggio C, focalizzando la nostra attenzione sulle macro di base e analizzandone a fondo l'operatività. In questa sezione ci dedichiamo all'analisi delle macro parametriche, le quali sono spesso denominate macro funzionali. Queste macro si distinguono principalmente per la loro capacità di ricevere argomenti in input, similmente a quanto avviene con le funzioni.

La struttura generale che si utilizza per stabilire la definizione di una macro di questo tipo è rappresentata dalla seguente struttura sintattica:

Macro Parametrica

Una Macro Parametrica, in linguaggio C, è una macro che accetta in ingresso degli argomenti. Tali argomenti non sono soggetti alle restrizioni di un tipo particolare ma sono, di fatto, capaci di rappresentare qualsiasi frammento sintattico.

```
#define NOME_DELLA_MACRO(arg1, arg2, ..., argN) testo_macro
```

Una macro parametrica è progettata per accettare uno o più argomenti, che devono essere separati tramite virgole. Questi ultimi diventano strumenti essenziali all'interno del corpo della macro per produrre porzioni di codice la cui personalizzazione dipende dai valori immessi. Quando il preprocessore si imbatte nella definizione di una macro che include argomenti, esegue un processo di memorizzazione simile a quello delle macro semplici, registrando il nome della macro insieme al suo rispettivo corpo. In fase di espansione, il preprocessore sostituisce l'istanza del nome della macro con il corpo memorizzato e, contemporaneamente, rimpiazza i parametri della macro con i valori specifici forniti. La distinzione fondamentale rispetto a una funzione risiede nel fatto che gli argomenti di una macro non sono soggetti alle restrizioni di un tipo particolare, essendo, di fatto, capaci di rappresentare qualsiasi frammento sintattico.

Facciamo un esempio: supponiamo di voler realizzare una macro parametrica che provoca l'uscita dal programma, qualora una data funzione, passata in input anch'essa, restituisca un valore pari a 0. Possiamo scrivere la macro in questo modo:

```
#define ESCI_SE_ZERO(f, x) ((f(x)) == (0) ? (exit(1)) : (0))
```

Si consideri il seguente frammento di codice:

```
#include <stdlib.h>
#define ESCI_SE_ZERO(f, x) ((f(x)) == (0) ? (exit(1)) : (0))

int main(int argc, char const *argv[]) {
    ESCI_SE_ZERO(atoi, argv[1]);
    return 0;
}
```

La macro serve a controllare se il numero fornito come primo parametro del programma è pari a zero. Se lo è, il programma termina restituendo un errore (1); altrimenti, restituisce 0 (programma terminato correttamente). La macro prende in input la funzione da usare (atoi, che converte una stringa in un intero) e l'argomento da passare a questa funzione (argv[1]).

Successivamente al passaggio del preprocessore, il codice si trasformerà in:

```
[ CONTENUTO DI stdlib.h ]

int main(int argc, char const *argv[]) {
    ((atoi(argv[1])) == (0) ? (exit(1)) : (0));
    return 0;
}
```

Nota bene: Analizzando la definizione di una macro, la quantità di parentesi utilizzate può apparire inusuale. Effettivamente, nella creazione di una macro si devono rispettare due norme fondamentali.

Prima regola: quando un parametro è presente nel corpo di una macro, è sempre consigliabile racchiuderlo all'interno di parentesi. Omettendo questo passaggio, si può incorrere in esiti indesiderati. Consideriamo un

Regole per Macro Parametrica

Quando si scrive una macro, parametrica o meno, in linguaggio C, conviene sempre seguire le seguenti regole per evitare errori nascosti ed effetti indesiderati:

- Racchiudere i parametri tra parentesi: se il corpo di una macro contiene uno dei parametri, è sempre meglio racchiuderlo tra parentesi per evitare errori di valutazione.
- Racchiudere tutto il corpo tra parentesi: se il corpo di una macro contiene uno o più operatori, è sempre meglio racchiudere tutto il corpo tra parentesi per evitare errori di precedenza.

esempio specifico. Supponiamo di definire una macro chiamata `DOUBLE`, il cui compito è calcolare il doppio di un dato numero, passato come argomento.

```
#define DOUBLE(x) x * 2
```

Nonostante una definizione di questo tipo possa sembrare chiara, cela un'insidia. Infatti, se impieghiamo la macro `DOUBLE` nel seguente modo:

```
int valore = 5;
int risultato = DOUBLE(valore + 1);
```

saremo propensi a pensare che, utilizzando la macro `DOUBLE`, il valore di risultato sia 12. Ossia, che la macro calcoli il doppio di 6. In realtà non è così. Il valore di risultato sarà, invece, 7. Questo accade perché la macro `DOUBLE` è stata espansa in:

```
int risultato = valore + 1 * 2;
```

Quindi l'espressione sarà valutata come $5 + 1 * 2$, ovvero 7, in quanto l'operatore di moltiplicazione `*` ha una precedenza maggiore rispetto all'operatore di somma `+`. Per risolvere questo problema, dobbiamo racchiudere i parametri tra parentesi. La definizione corretta della macro `DOUBLE` è:

```
#define DOUBLE(x) (x) * 2
```

Così facendo, in fase di espansione della macro, il codice diventerà:

```
int risultato = (valore + 1) * 2;
```

e il valore di risultato sarà 12, come ci aspettavamo.

Seconda regola: se il corpo di una macro contiene uno o più operatori, conviene sempre racchiudere tutto il corpo tra parentesi. Ritorniamo all'esempio di prima della macro `DOUBLE`. Supponiamo volere calcolare, sfruttando `DOUBLE`, il reciproco del DOPPIO di un numero: Potremmo scrivere il codice in questo modo:

```
#define DOUBLE(x) (x) * 2
...
double x = atoi(argv[1]);
double r = 1 / DOUBLE(x);
```

Il problema è che, con questo codice, non otterremo mai ciò che ci aspettiamo. Infatti, la macro `DOUBLE` verrà espansa in:

```
double r = 1 / x * 2;
```

Tuttavia, poiché l'operatore di divisione `/` ha la stessa precedenza dell'operatore di moltiplicazione `*`, l'espressione verrà valutata come:

$$r \leftarrow \frac{1}{x} \cdot 2$$

Pertanto il valore di r sarà $\frac{2}{x}$.

La versione corretta del codice di sopra si ottiene racchiudendo tutto il corpo della macro tra parentesi:

```
#define DOUBLE(x) ((x) * 2)
```


In questo modo l'espressione verrà valutata correttamente come:

$$r \leftarrow \frac{1}{x \cdot 2}$$

poichè l'espansione della macro in questo caso risulta:

```
double r = 1 / ((x) * 2);
```

e di conseguenza il valore di r sarà il reciproco del doppio di x , come ci aspettavamo e come in effetti avremmo voluto. In generale, quindi, è sempre meglio essere prudenti e racchiudere tra parentesi sia gli argomenti, sia l'intero corpo della macro.

3.5.1 Macro con espressioni multiple

Un utilizzo molto interessante delle macro è la combinazione di espressioni multiple. Questo significa che, all'interno del corpo di una macro, possiamo scrivere più espressioni, separate dall'operatore virgola.

Ad esempio supponiamo di voler realizzare una macro, chiamata GET_AGE, che mostra all'utente un prompt e legge un valore int da tastiera. La definizione potrebbe essere la seguente:

```
#define GET_AGE(anni) \
    (printf("Quanti anni hai: "), scanf("%d", &anni))
```

Nella macro, entrambe le chiamate a printf e scanf sono espressioni; pertanto, possono essere unite usando l'operatore virgola*.

Osserviamo che si sono impiegate le parentesi tonde per racchiudere il contenuto della macro. Questo è fattibile poichè la macro intera costituisce un'espressione singola.

In situazioni del genere, sebbene non sia sempre la scelta più raccomandata a causa del possibile impatto negativo sulla leggibilità del codice, si può optare per l'uso dell'operatore virgola. Tale approccio è applicabile anche in scenari leggermente più articolati, come quello illustrato qui sotto:

```
#define CHECK_AGE(anni) ( \
    printf("Quanti anni hai: "), \
    scanf("%d", &anni), \
    (anni < 18 ? (printf("Sei minorenne\n"), exit(1), 0) : 0) \
)
...
int main () {
    int anni;
    do
        CHECK_AGE(anni);
    while (anni < 18);
}
```

In alcuni casi, non risulta possibile (o efficiente) racchiudere tutto il codice in un'unica espressione. Ad esempio, qualora si decidesse di usare l'istruzione return anziché richiamare la funzione exit, questo metodo non si rivelerebbe adatto per la creazione della macro.

Macro con espressioni multiple

Quando si sviluppa una macro in linguaggio C, si ha la possibilità di includere più dichiarazioni nel corpo della stessa, utilizzando l'operatore virgola per concatenarle. È fondamentale avvolgere tali dichiarazioni fra parentesi tonde per prevenire eventuali errori durante la fase di compilazione.

* Per maggiori informazioni si faccia riferimento alla documentazione relativa all'operatore virgola

Avremmo potuto impiegare parentesi graffe, tuttavia questo avrebbe causato dei problemi. Infatti, se la macro fosse stata scritta così:

```
#define CHECK_AGE(anni) { \
    printf("Quanti anni hai: "); \
    scanf("%d", &anni); \
    if(anni < 18) {printf("Eta insufficiente\n"); return 1;} else
    {return 0;} \
}
...
int main (int argc, char *argv[]) {
    int anni;
    do
        CHECK_AGE(anni);
    while(anni < 18);
}
```

L'espansione genera un codice che non è sintatticamente valido. Otterremmo un errore di compilazione perché la macro si espanderebbe nel frammento di codice seguente:

```
do
{
    printf("Quanti anni hai: ");
    scanf("%d", &anni);
    if(anni < 18) {
        printf("Eta insufficiente\n");
        return 1;
    } else {
        return 0;
    }
};
while(anni < 18);
```

Il problema risiede nel punto e virgola ";" (in rosso) alla conclusione del blocco della macro, che spezza l'istruzione `do...while`. Di conseguenza, il compilatore interpreta il `while` come indipendente dal precedente `do`, causando un errore. Ovviamente avremmo potuto ovviare a tale problema semplicemente evitando di inserire il punto e virgola dopo la macro. Tuttavia, il codice della macro sarebbe apparso incoerente rispetto agli altri statement del programma.

3.5.2 Macro con istruzioni multiple

Le considerazioni fatte finora introducono le difficoltà che nascono quando in un'unica macro si vogliono eseguire più istruzioni. In questa situazione, l'uso dell'operatore virgola sarebbe inefficace proprio perché non è in grado di unire più istruzioni.

Soluzione con `do...while` Per risolvere il problema e garantire che la macro si comporti come un'istruzione unica in qualsiasi contesto, possiamo usare un ciclo `do...while`. A titolo di esempio si consideri il seguente frammento di codice:

```
#define CHECK_AGE(anni) \
do { \
    printf("Quanti anni hai: "); \
```

```
scanf("%d", &anni); \
if(anni < 18) { printf("Sei minorenne\n");} else {return 1;} \
} while(0)
```

Con questa versione, la macro può essere usata in qualsiasi contesto senza problemi:

```
int main () {
    int anni;
    do
        CHECK_AGE(anni);
    while (anni < 18);
}
```

Il ciclo `do...while` garantisce che la macro sia sempre trattata come un'unica istruzione, preservando la correttezza sintattica. Oltre a risolvere il problema della sintassi, possiamo arricchire la macro per includere più operazioni, ad esempio si consideri il seguente frammento di codice nel quale due `if` in cascata si assicurano di rendere pari e positivo il numero passato come argomento alla macro:

```
#define RENDI_POSITIVO_E_PARI(numero) \
do { \
    if ((numero) < 0) { \
        (numero) = -(numero); \
    } \
    if ((numero) % 2 != 0) { \
        (numero)++; \
    } \
} while (0)
```

Esempio di utilizzo:

```
int main() {
    int x = -5;

    RENDI_POSITIVO_E_PARI(x);
    printf("x ora vale: %d\n", x); // Stampa: x ora vale 6
    return 0;
}
```

Espansione della macro:

```
int main() {
    int x = -5;

    do {
        if ((x) < 0) {
            (x) = -(x);
        }
        if ((x) % 2 != 0) {
            (x)++;
        }
    } while (0);

    printf("x ora vale: %d\n", x);
    return 0;
}
```

In questo modo, la macro è sicura, flessibile e funziona correttamente in qualsiasi contesto.

3.5.3 Attenzione agli spazi prima delle parentesi

Sebbene la sintassi per definire una macro parametrica sia molto simile a quella per definire una funzione, esiste un dettaglio a cui prestare la *massima attenzione*: non bisogna mai inserire spazi tra il nome della macro e la parentesi di sinistra. A titolo di esempio si consideri il codice seguente, notando come uno spazio tra il nome della macro e la parentesi può causare problemi:

```
#define PRODOTTO (x, y) ((x) * (y))
```

In tal caso, infatti, il precompilatore considererà come corpo della macro la stringa $(x, y) ((x) \cdot (y))$ e non $((x) \cdot (y))$ come invece ci si aspetterebbe. La conseguenza di questo errore è che, quando si utilizzerà la macro SOMMA, il precompilatore sostituirà `PRODOTTO(5, 6)` con `((5) + (6))(5, 6)` e non con `((5) + (6))` come ci si aspetterebbe. La versione corretta è, invece, la seguente:

```
#define PRODOTTO(x, y) ((x) + (y))
```

3.5.4 Adozione di uno stile simil-funzionale

Si noti bene che, per definire le macro parametriche, non è necessario specificare una lista di parametri. Ad esempio, una definizione del genere è perfettamente legale:

```
#define TASSALO() (printf("Un Fiorino!\n"))
```

Quando vogliamo richiamare questa macro nel nostro codice, possiamo scrivere semplicemente:

```
TASSALO();
```

A prima vista, l'introduzione di una sintassi che permetta di definire una macro parametrica senza parametri può sembrare inutile. Avremmo potuto, ad esempio, usare una macro semplice. In realtà, si tratta di una scelta di consistenza con la sintassi di una funzione; dal punto di vista dell'utilizzatore, rende chiaro il fatto che si tratta di una macro che esegue operazioni, come una funzione, se la sintassi è simile a quella di una funzione.

3.6 Macro annidate

Le macro annidate in C rappresentano un potente strumento del pre-processore che consente di combinare più macro per creare definizioni più espressive e riutilizzabili. Questo approccio è utile quando si desidera incapsulare logica complessa in un'unica macro, mantenendo il codice leggibile e modulare.

3.6.1 Nessuna Ricorsione

Va comunque notato che le macro in C non supportano la ricorsione, perché il preprocessore esegue una sostituzione testuale diretta senza valutare il contesto.

```
// Errore!
#define FATTORIALE(n) (n == 0 ? 1 : (n * FATTORIALE(n - 1)))
```

Il preprocessore non può gestire chiamate ricorsive perché esegue una sostituzione diretta del testo, senza una fase di valutazione o controllo dei valori.

Listato 3.4: Esempio di macro non corretta

3.6.2 Un esempio corretto

```
1 #include <stdio.h>
2 #define QUADRATO(x) ((x) * (x))
3 #define DOPPIO_QUADRATO(x) (2 * QUADRATO(x))
4
5 int main(){
6     int x = 3;
7     printf("Il quadrato di %d e %d\n", x, QUADRATO(x));
8     printf("Il doppio del quadrato di %d e %d\n", x,
9         DOPPIO_QUADRATO(x));
10    return 0;
11 }
```

Listato 3.5: Esempio di macro annidata

Parte II

PROGRAMMAZIONE CONCORRENTE CON C11

Programmazione concorrente con C11: Ciclo di vita dei thread

4

4.1 Programmazione concorrente (in a nutshell)

Un'analisi approfondita della teoria della programmazione concorrente non rientra nell'obiettivo principale di queste note, tuttavia in questo capitolo intendiamo fornire una prima introduzione ai concetti chiave ad essa associati. Verranno richiamati in modo sintetico i motivi che ne supportano l'uso, e verranno riassunte in maniera concisa le principali sfide incontrate nell'area.

Il concetto di programmazione concorrente implica una strategia nella progettazione e sviluppo di software che permette l'esecuzione simultanea di diversi processi. Questo paradigma sfrutta al massimo le capacità di elaborazione contemporanee per aumentare notevolmente le prestazioni e l'efficienza. Questo approccio consente a vari processi* o thread di funzionare simultaneamente, agevolando così la gestione di compiti complessi o quelli che necessitano di risorse computazionali significative, rendendola una strategia valida per superare queste problematiche.

La programmazione concorrente trova largo impiego nei contesti in cui è fondamentale condividere risorse, raggiungere alti livelli di prestazione o ottimizzare l'efficacia delle operazioni di input e output.

Un classico esempio di programmazione concorrente è rappresentato dalle applicazioni server. Un server web, per illustrare, ha la capacità di lanciare una nuova attività autonoma per ogni richiesta cliente ricevuta. In questo modo, le richieste vengono gestite in parallelo, evitando così l'attesa per il completamento di una richiesta prima di procedere con l'elaborazione di un'altra. Questo metodo può incrementare in maniera significativa la reattività e l'efficienza del server.

Tuttavia, affrontare la programmazione concorrente comporta diverse sfide. La gestione della sincronizzazione e il coordinamento tra le varie attività sono complessi e possono dar luogo a un numero considerevole di problematiche, come ad esempio race condition, deadlock, livelock, starvation. Di conseguenza, è cruciale applicare tecniche di sincronizzazione con l'utilizzo di strumenti idonei per assicurarsi che il software concorrente funzioni in modo corretto e sia affidabile.

Per comprendere il concetto di programmazione concorrente, è fondamentale distinguere i due principali strumenti per l'implementazione della programmazione concorrente: i processi e i thread.

Nel primo, diversi processi indipendenti condividono le risorse del sistema e comunicano tra loro attraverso meccanismi di comunicazione interprocesso (IPC) come pipe, code messaggi o memoria condivisa. Nel secondo, un singolo processo contiene più thread che condividono lo stesso spazio di indirizzi, facilitando la condivisione di dati e l'interazione.

* in seguito, e in modo più approfondito in una sezione successiva del documento, esamineremo il concetto di processo; per ora consideriamolo come un programma attivo su un dato sistema

4.1	Programmazione concorrente (in a nutshell)	29
4.1.1	Più tipi di Thread	31
4.1.2	Thread e memoria	33
4.2	Supporto ad applicazioni concorrenti in C11	33
4.3	Threading: elementi di base	34
4.3.1	La creazione: <code>thrd_create</code>	34
4.3.2	L'identificazione: <code>thrd_current</code>	35
4.3.3	L'unione, l'attesa: <code>thrd_join</code>	36
4.3.4	L'indipendenza: <code>thrd_detach</code>	37
4.3.5	La terminazione: <code>thrd_exit</code>	37

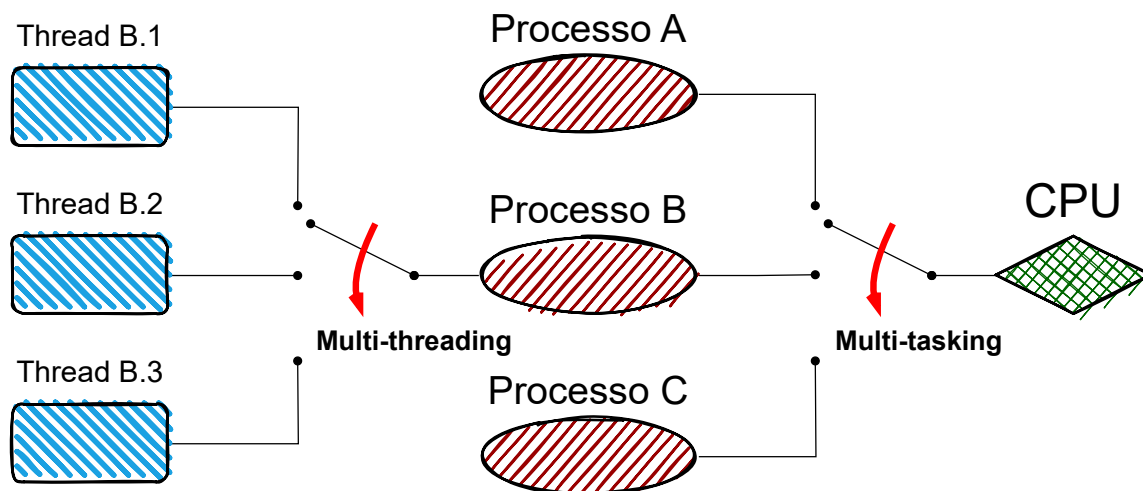


Figura 4.1: Processi e Thread

1: alcuni di questi saranno presentati più avanti nella dispensa

Processo

Un processo rappresenta un'entità autonoma in esecuzione, caratterizzata da un proprio spazio di memoria. Ogni processo mantiene uno spazio di memoria unico e distinto. La comunicazione fra processi richiede l'impiego di meccanismi appropriati, quali pipe, code di messaggi o memoria condivisa¹. Tali metodologie sono impiegate nelle applicazioni dove è fondamentale l'isolamento tra diverse unità di lavoro, specialmente per ragioni di sicurezza informatica e protezione.

Thread

Un thread rappresenta l'elemento fondamentale di esecuzione all'interno di un dato processo. Poiché i thread appartenenti allo stesso processo fanno uso dello stesso indirizzo di memoria, comunicare tra essi è notevolmente facilitato, dato che possono accedere direttamente a identiche variabili e strutture dati. Gli scenari tipici in cui si utilizzerebbero i thread includono sistemi in cui sono necessarie molteplici operazioni leggere e simultanee, come nel caso della gestione delle richieste su un server web.

Perché distinguere? In generale, l'allocazione e il controllo dei processi tendono a consumare maggiori risorse rispetto ai thread. Anche se i processi garantiscono un isolamento superiore che minimizza il rischio di propagazione di errori o vulnerabilità tra diverse unità operative, la complessità della loro gestione aumenta notevolmente per implementazioni che richiedono concorrenza. Ma quali aspetti sono fondamentali da valutare nella decisione tra processi e thread?

Thread Quando si intende condividere dei dati presenti in memoria cercando di minimizzare sia l'overhead² che le complicazioni legate alle operazioni di condivisione.

2: L'overhead è il costo aggiuntivo necessario per eseguire un'operazione, oltre al lavoro principale che vogliamo svolgere. Quando lavoriamo con processi e thread, dobbiamo considerare che crearli e gestirli ha un costo aggiuntivo, che chiamiamo overhead.

Processi In situazioni in cui la separazione tra le singole unità operative risulta essenziale oppure quando l'applicazione deve garantire un altissimo livello di sicurezza ed affidabilità.

Essenzialmente, la distinzione tra thread e processi va oltre una semplice questione stilistica o accademica; essa ha un impatto significativo sulla progettazione e sull'efficienza complessiva di un sistema che gestisce operazioni concorrenti.

Caratteristica	Processo	Thread
Spazio di memoria	Separato	Condiviso
Isolamento	Elevato	Minimo
Comunicazione	Complessa	Semplice (memoria condivisa)
Costo di creazione	Alto	Basso
Cambio di contesto	Costoso	Leggero

Tabella 4.1: Processi e Thread

4.1.1 Più tipi di Thread

Tutte le diverse tipologie di thread esistenti, sono state create per affrontare l'elevata pesantezza dei processi tradizionali. Un thread, noto anche come "processo leggero", si configura come un percorso di esecuzione interna a un processo.

Sebbene i thread condividano congiuntamente lo stesso spazio di memoria del processo principale, essi mantengono singolarmente il proprio stack e registro di esecuzione individuali.

Le principali categorie di thread

I thread possono essere classificati in alcune categorie differenti. Anche se condividono tra loro gli stessi elementi essenziali, il concetto di thread può essere realizzato attraverso diversi metodi e strategie di implementazione, sensibilmente differenti l'uno dall'altro.

Tre principali tipologie di thread

- ▶ User-Level Thread
- ▶ Kernel-Level Thread
- ▶ Hybrid Thread

Thread a livello utente (User-Level Threads, ULT) I thread vengono amministrati autonomamente dalla libreria dei thread, senza che il sistema operativo sia coinvolto o addirittura consapevole della loro presenza.

È l'applicazione stessa che si occupa dell'ideazione, delle operazioni di sincronizzazione e della pianificazione dei thread.

Un vantaggio di questo approccio è che può essere molto leggero in termini di overhead, poiché non c'è necessità di operazioni costose legate al kernel.

Tuttavia, presenta anche delle limitazioni, come il fatto che il sistema operativo non è in grado di gestire i thread in modo indipendente o ottimale, specialmente quando si tratta di sfruttare più core della CPU.

Cambio di contesto

Il cambio di contesto avviene quando il processore smette di eseguire un processo o un thread per passare all'esecuzione di un altro. È come se un insegnante passasse da correggere un compito a correggerne un altro: deve segnare dove è arrivato, mettere da parte il primo compito e prendere il secondo. Il sistema operativo (OS) deve salvare lo stato del processo o thread attuale e caricare lo stato di quello successivo. Questo implica:

- ▶ Salvare nei registri della CPU lo stato del processo attuale (program counter, registri di uso generale, stack pointer, ...).
- ▶ Aggiornare la tabella dei processi/thread con le informazioni sul processo sospeso.
- ▶ Caricare lo stato del nuovo processo nei registri della CPU.
- ▶ Passare il controllo alla nuova istruzione del nuovo processo o thread.

Questo lavoro richiede tempo e introduce un overhead.

Primi esempi pratici

Thoth (1976): Uno dei primi sistemi a introdurre il concetto di thread. UNIX (anni '80): I thread diventano uno strumento concreto nei sistemi UNIX attraverso librerie come pthreads (POSIX threads). Mach (fine anni '80): Un sistema operativo microkernel che enfatizza l'uso dei thread.

Inoltre, se un thread è bloccato (ad esempio, in attesa di I/O), può bloccare l'intero processo, rendendo difficile l'utilizzo efficace delle risorse multiple.

✓ Vantaggi

- ▶ **Switch rapido tra thread:** Il cambio di contesto è molto veloce perché non richiede intervento del kernel.
- ▶ **Portabilità:** Possono funzionare su qualsiasi sistema operativo che supporti i processi.
- ▶ **Flessibilità:** L'applicazione può implementare algoritmi di scheduling ottimizzati per i suoi bisogni.

✗ Svantaggi

- ▶ **Blocco totale del processo:** Se un thread a livello utente effettua una chiamata di sistema bloccante, l'intero processo si blocca.
- ▶ **Mancanza di parallelismo vero:** Anche su sistemi multi-core, i thread utente di un singolo processo non possono sfruttare più core contemporaneamente.

Implementazioni ULT

- ▶ **Pthread ULT** (su modalità user-level in alcuni sistemi)
- ▶ **Green Threads** (Java fino a JDK 1.2)
- ▶ **Fiber** piccole unità di esecuzione che possono essere pianificate, sospese e riprese dall'applicazione, senza l'intervento diretto del sistema operativo

Thread a livello kernel (Kernel-Level Threads, KLT) La gestione è direttamente a carico del kernel del sistema operativo. I thread sono tutti riconoscibili dal sistema operativo, che effettua lo scheduling e si occupa della loro amministrazione.

✓ Vantaggi

- ▶ **Parallelismo reale:** I thread possono essere eseguiti su core diversi contemporaneamente.
- ▶ **Chiamate di sistema bloccanti isolate:** Se un thread si blocca su una chiamata di sistema, gli altri possono continuare a eseguire.
- ▶ **Supporto nativo:** Il kernel fornisce supporto diretto per thread e sincronizzazione.

✗ Svantaggi

- ▶ **Cambio di contesto più costoso:** Il passaggio di contesto richiede intervento del kernel, rendendolo più pesante rispetto ai thread utente.
- ▶ **Minore scalabilità:** Il kernel può avere difficoltà nel gestire un numero elevato di thread.

Implementazioni KLT

- ▶ **Linux Threads** (pthreads in modalità kernel-level)
- ▶ **Windows Threads** (API nativa di Windows per i thread)

Thread ibridi (Hybrid Threads) Thread a livello utente e thread a livello kernel sono integrati. Un gruppo di thread utente è associato a un gruppo di thread kernel. Di solito, si adotta il modello M:N, dove M rappresenta il numero di thread utente mappati su N thread kernel.

✓ Vantaggi

- ▶ **Flessibilità:** Permettono di ottenere i vantaggi sia dei thread utente che di quelli kernel.
- ▶ **Prestazioni ottimizzate:** Possono sfruttare il parallelismo dei thread kernel e la leggerezza dei thread utente.

✗ Svantaggi

- **Complessità di implementazione:** La gestione ibrida richiede algoritmi di scheduling sofisticati.
- **Overhead di mappatura:** L'overhead della mappatura M:N può essere significativo.

Alcune considerazioni Nei sistemi attuali, si preferisce generalmente l'impiego di thread a livello di kernel o thread ibridi, poiché risultano più facili da implementare e garantiscono maggiore prevedibilità nelle prestazioni. In determinati contesti applicativi specifici, come quelli inerenti gli ambienti embedded o i runtime di alcuni linguaggi particolari, i thread gestiti a livello utente emergono come un caso di studio interessante. Tali thread sono utilizzati per incrementare l'efficienza di specifiche operazioni e costituiscono un'opportunità significativa per l'ottimizzazione nel campo dello sviluppo software.

4.1.2 Thread e memoria

Come già sottolineato in precedenza, i thread all'interno di un processo condividono lo stesso spazio di indirizzamento della memoria, ma ognuno ha il proprio stack. Lo stack di un thread viene utilizzato per memorizzare le variabili locali, i dati utili al controllo di flusso e le chiamate di funzione.

Durante l'esecuzione di un programma, il puntatore dello stack si sposta per gestire l'allocazione e la deallocazione della memoria. Contrariamente a questo, le aree di memoria per heap, codice e dati sono condivise tra i thread[†]. Tuttavia, ogni thread ha un proprio stack individuale che registra in modo indipendente le chiamate di funzione e gli stati.

4.2 Supporto ad applicazioni concorrenti in C11

La novità più significativa di C11 è il suo avanzato supporto alla concorrenza. Questo linguaggio include diverse primitive e funzioni essenziali, tra le quali alcune assumono un ruolo di particolare rilievo:

Threading come già detto in precedenza, un thread è una sequenza di controllo che può essere eseguita in parallelo ad altre all'interno dello stesso programma. C11 introduce una libreria per la gestione dei thread (`threads.h`), che fornisce funzioni per la creazione, il controllo e la sincronizzazione dei thread. Questo consente un miglior supporto alla programmazione concorrente, sfruttando le capacità dei sistemi multicore.

Mutua esclusione un mutex (abbreviazione di "mutua esclusione") è un meccanismo di sincronizzazione utilizzato per prevenire l'accesso simultaneo a una risorsa condivisa, come un'area di memoria, da parte di più thread. Utilizzando un mutex, solo un thread alla volta può accedere alla risorsa protetta, garantendo così la coerenza dei

Implementazioni HT

- **Solaris Threads** (modello M:N)
- **Windows Fibers** (un esempio di cooperazione tra thread kernel e thread utente)
- **Moderni runtime di Python e Java** (a volte usano modelli ibridi)

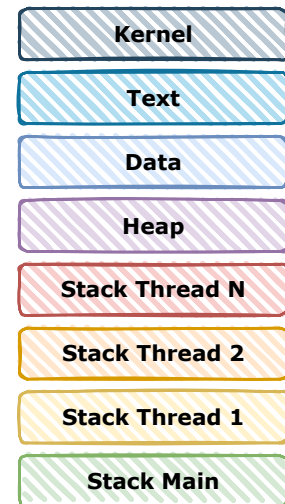


Figura 4.2: Virtual Memory Mapping

C11 (formalmente ISO/IEC 9899:2011) è uno degli standard del linguaggio di programmazione C. Ha rimpiazzato C99 (ISO/IEC 9899:1999). C11 è stato successivamente rimpiazzato da C17 (standard ISO/IEC 9899:2018, che si è limitato a correggerne alcuni errori) ed infine da C23 (ISO/IEC 9899:2024), che al momento della stesura di queste note non risulta ancora diffusamente supportato dai principali compilatori.

[†] In realtà, esistono anche i Thread Specific Storage (TSS), un sistema che permette a ciascun thread di un'applicazione di avere la propria istanza di una variabile, eliminando così la necessità di meccanismi come i blocchi per evitare situazioni di race condition. Questo aspetto sarà approfondito in seguito.

dati. Il mutex deve essere bloccato prima di accedere alla risorsa e sbloccato dopo che l'accesso è terminato.

Variabili di Condizione In C11, una “variabile di condizione” permette ai thread di rimanere in attesa fino a quando non si verifica una specifica condizione. Queste variabili sono utilizzate insieme ai mutex per gestire e coordinare il flusso di esecuzione tra i thread.

Thread-specific storage In C11, il thread-specific storage (TSS) permette di assegnare delle variabili che vengono mantenute separatamente per ciascun thread. Questo significa che ciascun thread ha la sua copia di tale variabile, isolata dalle copie degli altri thread. Per definire il TSS, si utilizza una keyword dedicata che garantisce che ogni thread abbia la propria versione della variabile. Questa caratteristica è utile per evitare interferenze tra i thread quando condividono risorse o variabili comuni.

Operazioni Atomiche In C11, le variabili atomiche forniscono un modo per gestire operazioni concorrenti sicure senza la necessità di blocchi espliciti. Sono definite nella libreria `stdatomic.h`, e consentono operazioni atomiche che garantiscono la coerenza dei dati quando accessi simultanei da più thread si verificano. Esse supportano operazioni atomiche su tipi base come `int` e possono essere utilizzate per evitare race condition.

Operazioni Atomiche

Un'operazione atomica è un'operazione eseguita dalla CPU in modo indivisibile, senza possibilità di interruzione da parte di altri thread o processi. Caratteristiche principali:

- ▶ Indivisibilità: l'operazione viene completata in un unico passo dal punto di vista di altri thread.
- ▶ Assenza di condizioni di gara: altri thread non possono vedere uno stato intermedio dell'operazione.
- ▶ Efficienza: spesso implementata tramite istruzioni hardware specifiche

4.3 Threading: elementi di base

Per introdurre gli elementi di base del supporto ai thread di C11 partiremo da due operazioni fondamentali: la creazione di thread e l'attesa della loro terminazione.

Passeremo poi a discutere l'opportunità del loro “distacco” dal flusso di esecuzione del programma principale e della loro terminazione. Nella presente sezione non tratteremo le primitive di sincronizzazione. Il nostro intento è invece rendere chiari i concetti fondamentali relativi all'intero ciclo di vita di un thread, partendo dal momento della sua creazione fino a quando giunge alla sua terminazione.

4.3.1 La creazione: `thrd_create`

La funzione `thrd_create` permette la creazione di nuovi thread. Questa funzione accetta tre argomenti: un puntatore a un descrittore di thread, un puntatore alla funzione che il thread eseguirà e un argomento per tale funzione.

Il prototipo della funzione è il seguente:

```
int thrd_create( thrd_t *thr, thrd_start_t func, void *arg );
```

Il programma riportato nel listato 4.1 mostra un esempio di base nel quale il flusso di esecuzione principale, il `main`, crea 5 thread, ognuno dei quali esegue la funzione `thread_func` (che si limita a stampare a video l'ID del thread).

Il `main`, successivamente alla creazione dei 5 thread, resta in attesa per la loro terminazione, utilizzando la funzione `thrd_join`.

```

1 #include <stdio.h>
2 #include <threads.h>
3
4 #define N 5 // numero di thread
5
6 int thread_func(void *arg){
7     thrd_t id = thrd_current(); // recupera l'id del thread
8     printf("Thread ID: %lu\n", id); // stampa l'ID del thread
9     return 0;
10 }
11
12 int main(){
13     thrd_t threads[N]; // array descrittori dei thread
14
15     for (int i = 0; i < N; i++){
16         thrd_create( // crea un thread
17             &threads[i], // id del thread
18             thread_func, // funzione del thread
19             NULL // nessun argomento alla funzione
20         );
21     }
22
23     for (int i = 0; i < N; i++) {
24         thrd_t ret = thrd_join(threads[i], NULL); // attende la terminazione del thread i
25         if (ret == thrd_success) { // verifica il risultato del join
26             printf("Thread con ID: %lu terminato correttamente.\n", threads[i]);
27         } else {
28             printf("Errore nel join del thread %d\n", i);
29         }
30     }
31
32     return 0;
33 }

```

Listato 4.1 Creazione Thread

4.3.2 L'identificazione: `thrd_current`

La funzione `thrd_current` in C11 permette di ottenere il descrittore del thread che sta eseguendo la chiamata. Questo è utile quando è necessario identificare il thread corrente in un programma multithreading.

```
thrd_t thrd_current(void);
```

Descrizione dei parametri e del valore di ritorno La funzione non accetta parametri. Restituisce un valore di tipo `thrd_t`, che rappresenta il thread chiamante. Il valore restituito da `thrd_current` può essere utilizzato per confrontare il thread attuale con altri thread, ad esempio per verificare se un determinato thread sta eseguendo una particolare operazione.

Nota:

- ▶ `thrd_current` è utile quando si vuole confrontare il thread chiamante con un altro thread noto.
- ▶ Non è opportuno utilizzare direttamente il valore restituito come indice o identificatore numerico senza conversioni, poiché il tipo `thrd_t` è definito in modo opaco.
- ▶ Il confronto tra due valori di `thrd_t` dovrebbe avvenire tramite la funzione `thrd_equal`, il cui prototipo è riportato di seguito.

```
int thrd_equal( thrd_t lhs, thrd_t rhs );
```


4.3.3 L'unione, l'attesa: `thrd_join`

La funzione `thrd_join` in C11 è utilizzata per sincronizzare un thread con il suo chiamante. In altre parole, permette a un thread *A* di aspettare che un thread *B* (il thread che è stato creato) termini la sua esecuzione prima di proseguire.

La sintassi di `thrd_join` è la seguente:

```
int thrd_join( thrd_t thr, int *res );
```

Dove:

thr è il descrittore del thread che deve essere “atteso” dal chiamante. È di tipo `thrd_t`, che è una struttura rappresentante un thread.

res è un puntatore a una variabile di tipo `int`, che può essere usata per ricevere il valore di ritorno del thread terminato. Se non si è interessati al risultato, è possibile passare `NULL` in luogo a `res`.

I possibili valori di ritorno della chiamata di `join` sono `thrd_success` se la chiamata ha avuto successo oppure `thrd_error` altrimenti. Gli ulteriori valori di stato e di errore definiti nella libreria di threading di C11 sono riportati nel listato 4.2.

Listato 4.2: Stati e codice errore

```
enum {

/* successo */
thrd_success = /* ... */,

/* errore di memoria */
thrd_nomem = /* ... */,

/* errore di timeout */
thrd_timeout = /* ... */,

/* risorsa indisponibile */
thrd_busy = /* ... */,

/* errore generico */
thrd_error = /* ... */

};
```

Il comportamento della `join` è indefinito se – il thread sul quale l'operazione è condotta – è stato precedentemente distaccato (più avanti approfondiremo il distaccamento – `detach` di un thread) o unito (`join`) a un altro thread.

Questi passaggi rappresentano i fondamentali imprescindibili per l'amministrazione dei thread in C11, fornendo un quadro strutturale complesso e dettagliato per il controllo e il coordinamento delle operazioni.

Perché facciamo la `thrd_join`?

Se i thread sono creati senza invocare successivamente la funzione `thrd_join`, potrebbero emergere complicazioni. Questi problemi sono spesso correlati a una gestione delle risorse di sistema che risulta inefficace o non deterministica, il che può alterare il comportamento previsto del programma in funzione.

Analizziamo attentamente le implicazioni di questa situazione:

Le risorse del thread non vengono liberate Al termine di un thread, è cruciale che il sistema operativo rilasci tempestivamente le risorse collegate, inclusa, per esempio, lo stack del thread. Qualora non si chiami la funzione `thrd_join`, può verificarsi un ritardo nel rilascio delle risorse oppure, talvolta, il sistema potrebbe non riuscire a rilasciarle affatto.

Completamento prematuro del programma Qualora il thread principale non eseguisse `thrd_join`, è possibile che esso termini la sua esecuzione prima che i thread da lui creati abbiano portato a termine il loro lavoro. Anche se tale situazione non rappresenta necessariamente un errore nel codice, può comunque causare scenari in cui il programma principale termina mentre i thread da esso generati sono ancora in esecuzione. In queste circostanze, esiste la possibilità che i thread creati siano interrotti

all'improvviso a causa della chiusura del thread principale, determinando così la fine dell'intero processo. Questo potrebbe comportare un funzionamento anomalo del programma, poiché alcune operazioni potrebbero non essere completate.

Impossibilità di controllare il risultato del thread Se non viene fatta l'operazione di join, il thread principale non ha modo di raccogliere il valore di ritorno del thread secondario. In altre parole, non potrà sapere se il thread è terminato correttamente o se ha restituito un errore.

4.3.4 L'indipendenza: `thrd_detach`

In situazioni in cui un thread agisce separatamente dal processo principale, l'isolamento dal contesto operativo generale può rivelarsi vantaggioso. Tale isolamento garantisce che il thread possa eseguire le proprie operazioni senza essere influenzato o interrotto dal resto del programma, permettendo quindi un'esecuzione più scorrevole e priva di conflitti.

Questo processo è comunemente identificato con il termine *detach*, ovvero *distacco*. Una volta che il thread è distaccato, il sistema operativo si incarica automaticamente della gestione e del rilascio delle risorse da esso impiegate al completamento dell'esecuzione del thread stesso.

La sintassi di `thrd_detach` è la seguente:

```
int thrd_detach( thrd_t thr );
```

L'operazione `thrd_detach` "distacca" il thread identificato da `thr` dall'ambiente corrente. Di fatto si tratta di un'alternativa alla `join`, da utilizzare quando il flusso di esecuzione principale e i flussi secondari possono procedere in modo del tutto indipendente.

4.3.5 La terminazione: `thrd_exit`

La funzione `thrd_exit` viene impiegata per concludere, in una maniera sicura e precisa, l'esecuzione di un thread. Assicura che tutte le risorse allocate durante il ciclo di vita del thread vengano liberate in modo appropriato. Questa funzione rappresenta un elemento essenziale nella gestione della durata e del ciclo vitale dei thread durante esecuzioni parallele.

```
void thrd_exit( int res );
```

Sostanzialmente, il metodo `thrd_exit` concluderà l'esecuzione del thread attualmente attivo e assegnerà un valore di ritorno specificato tramite il parametro `res`. Nel caso in cui il thread corrente rappresenti l'ultimo esistente nel contesto del programma in esecuzione, l'applicazione stessa giungerà a una conclusione definitiva, equivalendo all'invocazione della funzione `exit` con l'argomento `EXIT_SUCCESS`, segnando così un termine riuscito ed ordinato dell'esecuzione complessiva.

Quando la funzione che rappresenta il thread termina restituendo un valore (usando semplicemente `return`), ciò equivale a chiamare implicitamente `thrd_exit` con quel valore di uscita. In altre parole, se la thread function finisce "naturalmente" con un `return`, il valore ritornato verrà

Potrei, invece, evitare semplicemente l'operazione di join?

È legittimo domandarsi se anziché eseguire l'operazione di distacco non sia possibile semplicemente evitare di fare la `join` successivamente alla creazione del thread, lasciandolo "libero" di correre. Se da un lato si tratta di una possibilità concreta e possibile, non è certamente una scelta ottimale. Infatti, un thread né unito (`join`) né distaccato (`detach`) porta, al termine della propria esecuzione, all'inutile occupazione di preziose risorse di sistema, che non sarebbero rilasciate automaticamente, come invece avverrebbe se il thread fosse stato correttamente distaccato.

```

1  #include <stdio.h>
2  #include <threads.h>
3  #include <stdbool.h>
4  #include <stdlib.h>
5
6  #define N 5 // numero di thread
7
8  int thread_func(void *arg)
9  {
10     thrd_t id = thrd_current();           // recupera l'id del thread
11     int tempo_rnd = 1 + (rand() % 4);      // genera un tempo casuale tra 1 e 4 secondi
12     printf("Inizio Thread: %lu, resta attivo " // stampa l'inizio del thread
13           "per %d sec\n", id, tempo_rnd);
14     struct timespec ts =                  // imposta il tempo di attesa
15     { .tv_sec = tempo_rnd, .tv_nsec = 0 };
16     thrd_sleep(&ts, NULL);                // attende il tempo impostato
17     printf("Fine Thread: %lu\n", id);      // stampa la fine del thread
18     return thrd_success;                  // ritorna il valore di successo
19 }
20
21 int main() {
22     thrd_t threads[N];                    // array di thread
23
24     for (int i = 0; i < N; i++) {
25         thrd_create(                       // crea un thread
26             &threads[i],                  // id del thread
27             thread_func,                  // funzione del thread
28             NULL                           // argomento della funzione
29         );
30
31         printf("Thread %lu creato.\n", threads[i]);
32         fflush(stdout);
33
34         thrd_detach(threads[i]);           // imposta il thread come detached
35                                           // il thread non deve essere esplicitamente
36                                           // terminato il sistema operativo si occupa
37                                           // di liberare le risorse allocate
38     }
39
40     for (int i = 0; i < N; i++) {
41         int ret = thrd_join(threads[i], NULL); // attende la terminazione del thread i
42         if (ret == thrd_success)                // verifica il risultato del join
43             printf("Thread %lu terminato correttamente.\n", threads[i]);
44         else
45             printf("Errore nel join del thread %lu\n", threads[i]);
46     }
47
48     printf("Termine Programma.\n");
49     return 0;
50 }

```

Listato 4.3 Distacco Thread

usato come codice di uscita del thread, ed è equivalente a chiamare `thrd_exit` dal thread corrente.

Tuttavia, ci sono alcuni motivi per cui potresti voler chiamare esplicitamente `thrd_exit` anziché utilizzare `return`:

Uscita anticipata da profondità annidata: Se ti trovi in una funzione intermedia o in un contesto in cui non puoi semplicemente restituire (per esempio, in una funzione che non è la thread function stessa) e desideri terminare il thread immediatamente, puoi chiamare `thrd_exit` per interrompere l'esecuzione del thread corrente senza dover propagare il `return` lungo tutta la catena di chiamate.

Chiarezza semantica: In alcuni casi può essere utile chiamare esplicitamente `thrd_exit` per indicare in modo esplicito che stai terminando il thread, soprattutto se ciò avviene in una parte del codice dove l'uso di `return` potrebbe non rendere immediatamente chiaro che stai terminando il thread anziché semplicemente terminare una funzione.

Contesto in cui non si può usare `return`: Se sei all'interno di una funzione che non fa parte direttamente della thread function (ad esempio, in una funzione di callback o in un helper), non puoi utilizzare `return` per terminare il thread corrente; in questo caso, `thrd_exit` è l'unico modo per terminare il thread in modo ordinato.

Una volta invocata `thrd_exit`, il thread non riprenderà l'esecuzione: la chiamata termina definitivamente il thread corrente.

Programmazione concorrente con C11: Operazioni Atomiche e Lock

5

5.1 Premessa

Nel contesto della programmazione concorrente, uno degli aspetti fondamentali per garantire il corretto funzionamento di un sistema multi-threaded è la gestione della sincronizzazione tra i thread. In particolare, le operazioni atomiche e i meccanismi di lock sono strumenti essenziali per evitare race conditions, corruzione dei dati e comportamenti indesiderati.

Questo capitolo introduce il concetto di operazioni atomiche, spiegandone l'importanza e illustrando le primitive disponibili in C11. Successivamente, verranno presentati i meccanismi di lock, confrontando diverse strategie di sincronizzazione.

5.2 Operazioni Atomiche

5.2.1 Definizione di Atomicità

Un'operazione è definita atomica quando viene eseguita in modo tale da essere considerata un'unità indivisibile. Ciò significa che non c'è alcuna possibilità che un thread diverso possa osservare uno stato intermedio di tale operazione durante la sua esecuzione. L'atomicità è un concetto di cruciale importanza nei contesti di programmazione concorrente, in quanto assicura il mantenimento della coerenza dei dati condivisi tra più thread o processi, prevenendo situazioni in cui dati incoerenti possano compromettere il sistema.

Un esempio di operazione di incremento può essere rappresentato dal seguente pseudocodice:

1. 'load' - Carico il valore corrente di una variabile 'x'.
2. 'incr' - Incremento il valore di 'x' di 1.
3. 'store' - Memorizzo il nuovo valore di 'x' nella sua posizione originale.

Se nel corso di queste tre operazioni ('load', 'incr', 'store') il thread che le sta eseguendo venisse interrotto, e altri thread operassero sulla stessa variabile, si potrebbero verificare delle inconsistenze nei dati. Tuttavia, qualora fosse possibile considerare queste tre operazioni come una singola operazione atomica di incremento, dove l'incremento è attuato come un'entità indivisibile, si garantirebbe che nessun altro processo possa intervenire durante il calcolo, preservando così l'integrità delle operazioni.

5.1	Premessa	41
5.2	Operazioni Atomiche . .	41
5.2.1	Definizione di Atomicità	41
5.2.2	Supporto in C11	42
5.2.3	Classi di Memoria	
	Atomica	43
5.3	Meccanismi di Lock . . .	44
5.3.1	Spinlock	44

5.2.2 Supporto in C11

Lo standard C11 introduce il supporto per operazioni atomiche tramite l'header `<stdatomic.h>`. Questo consente di dichiarare variabili atomiche e di eseguire alcune operazioni su di esse senza bisogno di ulteriori meccanismi di sincronizzazione.

Esempio di dichiarazione di una variabile atomica:

```
#include <stdatomic.h>
atomic_int counter = 0;
```

Semplici operazioni condotte su variabili dichiarate atomiche sono garantite essere atomiche a loro volta. Tuttavia, quando si opera su variabili atomiche, è sempre preferibile utilizzare le opportune funzioni messe a disposizione da C11.

L'operazione di incremento atomico:

```
atomic_fetch_add(&counter, 1);
```

L'uso di queste funzioni garantisce che l'operazione sia eseguita in modo indivisibile, evitando race condition.

1. `C atomic_load (const volatile A * variabile)`: serve per leggere in modo atomico il valore di una variabile atomica, senza che altre operazioni o thread possano interrompere la lettura. L'argomento è un puntatore ad un tipo atomico volatile. Il tipo di ritorno è coerente col tipo di A.
2. `void atomic_store (volatile A * variabile, C valore)`: serve per scrivere in modo atomico il valore di una variabile atomica, senza che altre operazioni o thread possano interrompere la scrittura. Il primo argomento è un puntatore ad un tipo atomico volatile. Il secondo argomento (C) è un tipo non atomico coerente col tipo di A.
3. `C atomic_exchange (volatile A * variabile, C valore)`: sostituisce il valore di una variabile atomica con un nuovo valore e restituisce il valore precedente. Questa operazione è chiamata read-modify-write, perché legge il valore attuale della variabile, lo sostituisce con un nuovo valore e restituisce il valore che la variabile aveva prima della modifica.
4. `_Bool atomic_compare_exchange_strong (volatile A * var, C * valore_atteso, C nuovo_valore)`: viene usata per modificare il valore di una variabile atomica solo se il valore attuale corrisponde a quello atteso. È un'operazione fondamentale nella programmazione concorrente, perché permette di implementare meccanismi di sincronizzazione senza l'uso di lock. Confronta il valore attuale della variabile atomica (var) con valore_atteso. Se i due valori sono uguali a bit-a-bit, la variabile viene sostituita con nuovo_valore, ovvero realizza un'operazione di lettura-modifica-scrittura (read-modify-write). Se i due valori sono diversi, la funzione aggiorna valore_atteso con il valore attuale della variabile di fatto esercitando un'operazione di sola lettura (load). Esiste anche la versione "debole" di questa chiamata, `_Bool atomic_compare_exchange_weak (volatile A * var, C* valore_atteso, C valore)`, che può fallire spuriamente (cioè anche se i valori sono uguali), obbligando a riprovare

l'operazione in un ciclo. Quest'ultima versione (debole) è spesso più veloce su alcune architetture.

5. `C atomic_fetch_add (volatile A * var, M incr)`: La funzione esegue in modo atomico l'operazione di somma su una variabile atomica, sostituendone il valore con il risultato della somma tra il valore attuale e un valore specificato (`incr`). La funzione restituisce il valore che la variabile aveva prima della modifica. Essendo un'operazione di tipo `read-modify-write`, la funzione prima legge il valore corrente della variabile, poi lo modifica sommando `incr` e infine scrive il nuovo valore.
6. `C atomic_fetch_sub (volatile A * var, M decr)`: esegue in modo atomico l'operazione di sottrazione su una variabile atomica, sostituendone il valore con il risultato della sottrazione tra il valore attuale e un valore specificato (`decr`). La funzione restituisce il valore che la variabile aveva prima della modifica. Essendo un'operazione di tipo `read-modify-write`, la funzione prima legge il valore corrente della variabile, poi lo modifica sottraendo `decr` e infine scrive il nuovo valore.
7. `C atomic_fetch_and (volatile A * var, M altro)`: esegue in modo atomico un'operazione di AND bitwise tra il valore attuale di una variabile atomica e un valore specificato (`arg`). Il risultato dell'operazione sostituisce il valore precedente della variabile, mentre la funzione restituisce il valore originale della variabile, ossia quello che aveva prima della modifica. Poiché si tratta di un'operazione di tipo `read-modify-write`, la funzione prima legge il valore attuale della variabile, poi esegue l'operazione AND bitwise con `arg` e infine scrive il nuovo valore nella variabile.
8. `atomic_fetch_or` e `atomic_fetch_xor`: analoghe alla funzione descritta sopra, con la differenza che le operazioni considerate sono OR e XOR.

Tutte le funzioni atomiche standard del C11 hanno una versione con il suffisso `_explicit`, che permette di specificare esplicitamente il modello di memoria utilizzato per l'operazione. Nella versione base, il modello di memoria predefinito è `memory_order_seq_cst`, che garantisce la massima sincronizzazione tra i thread, ma in alcuni scenari è possibile ottenere migliori prestazioni scegliendo un modello meno restrittivo. Le versioni `_explicit` accettano un ulteriore parametro che può essere, ad esempio, `memory_order_relaxed` (senza garanzie di ordine), `memory_order_acquire` (per sincronizzare letture), `memory_order_release` (per sincronizzare scritture) o combinazioni più avanzate. Questo consente un controllo più fine sulle operazioni atomiche, adattandole alle esigenze specifiche del programma.

5.2.3 Classi di Memoria Atomica

C11 permette di specificare differenti ordini di memoria per le operazioni atomiche, influenzando il modo in cui gli accessi alla memoria vengono eseguiti:

`memory_order_relaxed` nessuna garanzia sull'ordine degli accessi
`memory_order_acquire` garantisce che tutte le operazioni precedenti nel thread siano visibili prima dell'accesso

memory_order_release garantisce che tutte le operazioni successive vedano gli aggiornamenti effettuati
memory_order_acq_rel combinazione di acquire e release
memory_order_seq_cst ordine sequenziale forte (default)

Quando si utilizza `memory_order_seq_cst`, tutte le operazioni atomiche appaiono eseguite in un ordine globale ben definito per tutti i thread, come se fossero serializzate in una sequenza unica. Questo garantisce che tutti i thread vedano le modifiche nello stesso ordine, evitando effetti di riorganizzazione delle istruzioni da parte del compilatore o dell'hardware.

In altre parole, `memory_order_seq_cst` è l'unico modello di memoria tra quelli disponibili in C11 che assicura la coerenza sequenziale (Sequential Consistency), ovvero la proprietà che Lamport ha formalizzato nel suo famoso articolo *"How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs"* (1979).

5.3 Meccanismi di Lock

Le variabili atomiche permettono di eseguire operazioni di lettura, scrittura e aggiornamento senza la necessità di utilizzare meccanismi di sincronizzazione più complessi, garantendo l'assenza di condizioni di gara per singole operazioni. Tuttavia, quando più operazioni atomiche devono essere eseguite in modo coordinato per garantire la coerenza di una struttura dati condivisa, l'uso delle sole variabili atomiche diventa insufficiente. In questi casi, è necessario introdurre primitive di sincronizzazione che impongano un ordine preciso nell'accesso alle risorse condivise. Una delle soluzioni più semplici ed efficienti è lo spinlock, un meccanismo di mutua esclusione basato su variabili atomiche, che consente a un thread di attendere attivamente fino a quando non ottiene l'accesso alla risorsa. Nella sezione successiva vedremo come implementare uno spinlock utilizzando le operazioni atomiche introdotte finora.

5.3.1 Spinlock

Un spinlock rappresenta un meccanismo di sincronizzazione progettato per consentire ai thread di accedere in modo esclusivo a una risorsa, come una variabile o un segmento di memoria. Questo è ottenuto attraverso la mutua esclusione, dove il thread rimane in uno stato di attività continua finché non acquisisce il lock desiderato.

Esempio:

```
while(atomic_flag_test_and_set(&lock)) { }
    // Sezione critica
atomic_flag_clear(&lock);
```


Esempio di Spinlock

Il programma riportato nel Listato 5.1 a pagina 46, scritto in linguaggio C11, genera 5 thread operanti in modo concorrente i quali impiegano uno spinlock, implementato tramite la struttura atomica `atomic_flag`, al fine di garantire una sincronizzazione sicura durante l'accesso a una sezione critica condivisa. Ogni singolo thread opera secondo il seguente ciclo iterativo: inizialmente acquisisce lo spinlock per ottenere un accesso esclusivo alla porzione critica del codice; successivamente, emula l'esecuzione di un carico computazionale per la durata di un secondo, dopodiché rilascia lo spinlock.

Completata questa fase, il thread si pone in uno stato di sospensione passiva, dormendo per un secondo, prima di riprendere il ciclo operativo. Questo processo viene iterato un numero di volte stabilito dalla costante `ITERATION`, fissata a 5.

Una volta che tutti i thread hanno completato le rispettive esecuzioni, il thread principale si pone in attesa della loro terminazione completa, per poi procedere alla stampa del valore finale calcolato dalla variabile globale denominata `val`.

```

1  #include <stdio.h>
2  #include <threads.h>
3  #include <stdatomic.h>
4
5  #define ITERATION 5
6
7  atomic_flag lock = ATOMIC_FLAG_INIT;           // inizializzazione statica
8  struct timespec ts = { .tv_sec = 1, .tv_nsec = 0 }; // 1 secondo
9  double val = 0.0;                             // variabile condivisa
10
11 void spinlock(){ while (atomic_flag_test_and_set(&lock)); } // spinlock
12 void spinunlock(){ atomic_flag_clear(&lock); } // rilascio spinlock
13
14 void loop_che_perde_tempo(int sec){
15     time_t start = time(NULL);                 // tempo iniziale
16     while (time(NULL) - start < sec)           // per sec secondi
17         for (int i = 0; i < 1000000; i++) val += 1.0 / (i + 1); // aggiungo un po' di lavoro a caso
18 }
19
20 int thread_func(void *arg){
21     for (size_t i = 0; i < ITERATION; i++){    // per ITERATION volte
22         spinlock();                          // acquisisco spinlock
23         int id = *(int *)arg;                // leggo l'id del thread
24         printf("Thread %d si mette a dormire.\n", id);
25         loop_che_perde_tempo(1);              // lavoro di 1 secondo
26         printf("Thread %d si e svegliato\n\n", id);
27         spinunlock();                        // rilascio spinlock
28         thrd_sleep(&ts, NULL);               // dormo per 1 secondo
29     }
30     return thrd_success;
31 }
32
33 int main(){
34     const int N = 5;                          // numero di thread
35     thrd_t threads[N];                        // array di thread
36
37     int thread_ids[] = {0, 1, 2, 3, 4};       // id dei thread
38
39     for (int i = 0; i < N; i++) {
40         thrd_create( &threads[i], thread_func, &thread_ids[i] ); // creo i thread
41     }
42
43     for (int i = 0; i < N; i++) {
44         int ret = thrd_join(threads[i], NULL); // aspetto la fine dei thread
45         if (ret == thrd_success) printf("Thread %d terminato correttamente.\n", i);
46         else printf("Errore nel join del thread %d\n", i);
47     }
48
49     printf("Termine. Valore calcolato: %f\n", val);
50     return 0;
51 }

```

Listato 5.1 Spinlock

Programmazione concorrente con C11: Mutex e Condition Variables

6

Nel capitolo precedente, abbiamo discusso l'importanza della gestione dell'accesso a risorse condivise nei programmi concorrenti. Tale gestione è essenziale per prevenire race condition e per assicurare la correttezza durante l'esecuzione.

È vero che l'uso di variabili atomiche consente di eseguire operazioni senza interferenze esterne; tuttavia, quando è necessario proteggere blocchi di codice critici più ampi o coordinare l'esecuzione simultanea di thread multipli, diviene fondamentale impiegare primitive di sincronizzazione più sofisticate.

In questo contesto, abbiamo già esplorato l'utilizzo delle variabili atomiche per implementare una spinlock. Questa soluzione, sebbene valida in molti casi, presenta lo svantaggio di consumare cicli macchina inutilmente.

In C11, la libreria `<threads.h>` fornisce meccanismi standard per la gestione della mutua esclusione (cioè l'accesso esclusivo a determinate risorse) e della sincronizzazione dei thread. In particolare:

- ▶ I mutex (`mtx_t`) consentono di garantire l'accesso esclusivo a una sezione critica, impedendo che più thread la eseguano simultaneamente.
- ▶ Le condition variables (`cnd_t`) permettono ai thread di sospendersi in attesa di un evento e di essere risvegliati quando una certa condizione è soddisfatta, facilitando la comunicazione tra thread.

In questo capitolo esploreremo il funzionamento dei mutex e delle condition variables, mostrando come impiegarli per risolvere problemi di sincronizzazione nei programmi concorrenti.

6.1 Mutex

I mutex (mutual exclusion) sono uno degli strumenti principali per garantire che solo un thread alla volta possa accedere a una sezione critica del programma. Essi impediscono che più thread possano modificare simultaneamente lo stesso dato o risorsa, evitando così race conditions. In C11, i mutex sono definiti nel file di intestazione `<threads.h>` e sono utilizzati principalmente per proteggere blocchi di codice che accedono a risorse condivise.

6.1.1 Dichiarazione e Inizializzazione di un Mutex

In C11, un mutex viene dichiarato come una variabile di tipo `mtx_t`. La sua inizializzazione avviene tramite la funzione `mtx_init`. L'inizializzazione è necessaria prima che il mutex possa essere utilizzato per la sincronizzazione tra thread. Una volta inizializzato, il mutex può essere liberamente utilizzato per proteggere sezioni critiche.

6.1	Mutex	47
6.1.1	Dichiarazione e Inizializzazione di un Mutex	47
6.1.2	Lock e Unlock su mutex	49
6.1.3	Deadlock e Gestione delle Risorse	49
6.2	Condition Variables	49
6.2.1	Condition Variables in C11	50
6.2.2	Spurious Wakeups	51
6.2.3	Esempio di Produttori-Consumatori con Condition Variables	52

```

mtx_t mutex;
if (mtx_init(&mutex, mtx_plain) != thrd_success) {
    // Gestione dell'errore
}

```

Nel presente esempio, `mtx_plain` rappresenta una delle modalità predefinite disponibili per l'inizializzazione di un mutex. Tra le altre modalità accessibili vi sono `mtx_recursive`, che è progettata per gestire mutex ricorsivi, e `mtx_timed`, la quale offre la funzionalità di timeout. È importante notare che durante l'inizializzazione di un mutex, l'utente ha la possibilità di selezionare tra differenti modalità operative, ognuna delle quali modifica il comportamento del mutex nei confronti dei thread che tentano di acquisirlo. `mtx_plain`, `mtx_recursive` e `mtx_timed` sono le modalità più utilizzate, ciascuna con attributi distinti che le rendono appropriate per particolari situazioni operative. La modalità `mtx_plain` è la più semplice e comune. Quando un mutex è inizializzato con questa modalità, si comporta come un mutex tradizionale. Ogni thread che cerca di acquisire il mutex verrà bloccato finché il mutex non viene rilasciato dal thread che lo possiede. Non sono previste particolarità, e il mutex viene rilasciato una sola volta quando il thread che l'ha acquisito termina l'esecuzione della sezione critica. Questa modalità è ideale quando il comportamento richiesto è quello di un accesso esclusivo senza necessità di gestioni particolarmente complesse. La modalità `mtx_recursive` permette al thread che ha acquisito un mutex di acquisirlo nuovamente senza bloccare se stesso. In pratica, un thread che detiene già un mutex ricorsivo può acquisirlo ripetutamente, ma dovrà rilasciarlo lo stesso numero di volte prima che altri thread possano acquisirlo. Questa modalità è utile quando un thread chiama una funzione che a sua volta richiede di acquisire il mutex. Senza un mutex ricorsivo, il thread finirebbe per bloccare se stesso, generando un deadlock. La modalità `mtx_timed` consente di acquisire un mutex con un timeout. Quando un thread tenta di acquisire un mutex con questa modalità, può specificare un tempo massimo da aspettare prima di rinunciare e restituire un errore se il mutex non viene acquisito entro il tempo stabilito. Questo è particolarmente utile quando un thread non può aspettare indefinitamente, ma deve continuare a lavorare o fare altre operazioni se il mutex non è disponibile in tempi ragionevoli.

Listato 6.1: mutex con timeout

```

1 #include <time.h>
2
3 mtx_t timed_mutex;
4 struct timespec ts;
5 clock_gettime(CLOCK_REALTIME, &ts); // Ottieni il tempo corrente
6 ts.tv_sec += 1; // Aggiungi 1 secondo al tempo corrente
7
8 if (mtx_init(&timed_mutex, mtx_timed) != thrd_success) {
9     // Gestione dell'errore
10 }
11
12 if (mtx_lock(&timed_mutex) == thrd_success) {
13     // Acquisito il mutex
14     mtx_unlock(&timed_mutex);
15 } else {
16     // Timeout, mutex non stato acquisito
17     printf("Timeout nel tentativo di acquisire il mutex.\n");
18 }

```

6.1.2 Lock e Unlock su mutex

Una volta che un mutex è stato inizializzato, i thread possono acquisirlo (lock) e rilasciarlo (unlock) per accedere alla sezione critica. La funzione `mtx_lock` viene utilizzata per acquisire il mutex, bloccando l'accesso ad altri thread. Se il mutex è già stato acquisito da un altro thread, il thread che chiama `mtx_lock` sarà bloccato finché il mutex non viene rilasciato.

```
mtx_lock(&mutex);
// Sezione critica
mtx_unlock(&mutex);
```

Dopo aver terminato la sezione critica, il thread deve sempre chiamare `mtx_unlock` per liberare il mutex, permettendo ad altri thread di acquisirlo.

6.1.3 Deadlock e Gestione delle Risorse

Un aspetto cruciale nell'impiego dei mutex è la prevenzione del deadlock. Questo fenomeno si presenta quando due o più thread entrano in uno stato di blocco reciproco. In tale situazione, ogni thread rimane in attesa che l'altro o gli altri thread coinvolti rilascino il mutex, determinando così un'impasse che non permette ad alcun thread di proseguire la propria esecuzione.

Un deadlock può manifestarsi, per esempio, quando due thread bloccano mutex in sequenze opposte o incrociate. Immaginiamo due thread: A e B. Il thread A mantiene il lock sul mutex `alpha` e nel frattempo, il thread B blocca il mutex `beta`. Nella fase successiva, A, già in possesso del lock su `alpha`, tenta di ottenere il lock anche su `beta`. Contemporaneamente, B, che possiede il lock sul mutex `beta`, cerca di bloccare il mutex `alpha`. Questa situazione porta a un potenziale blocco del sistema. Per diminuire il rischio di deadlock, è essenziale adottare alcune adeguate strategie, come:

- Garantire l'acquisizione ordinata dei mutex: nel caso in cui sia necessario ottenere più mutex, è essenziale che i thread li acquisiscano seguendo un ordine coerente lungo tutte le esecuzioni possibili del programma.
- Ridurre al minimo il tempo di detenzione dei mutex: il mantenimento prolungato di un mutex in stato di blocco aumenta la probabilità di impedire l'esecuzione fluida di altri thread, causandone il blocco.

6.2 Condition Variables

Nella sezione precedente, abbiamo esplorato le mutex, che permettono di proteggere sezioni critiche, garantendo l'accesso esclusivo a risorse condivise. Tuttavia, quando si desidera che un thread possa attendere il verificarsi di una condizione prima di proseguire, i mutex da soli non sono sufficienti. In questi casi, è necessario un meccanismo che consenta ai thread di sospendersi fino al verificarsi di un evento specifico.

Le condition variables (variabili di condizione) sono progettate per facilitare la sincronizzazione tra i thread, permettendo a un thread

di mettersi in attesa fino a quando una certa condizione non viene soddisfatta.

Tuttavia, è importante notare che le variabili di condizione vanno sempre utilizzate insieme ai mutex: un thread deve detenere un mutex per poter utilizzare una condition variable. Quando un thread chiama una funzione come `cond_wait()`, esso rilascia temporaneamente il mutex mentre è in attesa e lo riacquista quando viene risvegliato, assicurando così che altri thread possano accedere alla risorsa protetta nel frattempo.

In sintesi, i mutex e le condition variables lavorano insieme per consentire ai thread di sincronizzarsi in modo sicuro ed efficiente, gestendo sia l'accesso alle risorse condivise che la comunicazione tra i thread.

6.2.1 Condition Variables in C11

In C11, le variabili di condizione sono gestite tramite la libreria `<threads.h>`, che fornisce il tipo `cond_t` per dichiarare variabili di condizione. Le operazioni principali associate alle condition variables sono la funzione `cond_wait()`, che sospende un thread fino a che non viene risvegliato da un altro thread, e la funzione `cond_signal()` o `cond_broadcast()`, che svegliano uno o tutti i thread in attesa sulla condition variable.

Le variabili di condizione vengono usate in scenari in cui è necessario implementare meccanismi di attesa attiva o notifica tra thread. Ad esempio, un thread potrebbe attendere che un altro thread termini una computazione prima di poter proseguire. Oppure, un produttore potrebbe attendere che ci sia spazio nel buffer prima di inserire un nuovo dato, mentre un consumatore potrebbe aspettare che ci siano dati disponibili prima di prelevarli.

Comportamento di un Thread al Risveglio

Quando un thread chiama `cond_wait()`, esso rilascia temporaneamente il mutex associato alla variabile di condizione, entrando in uno stato di attesa. Una volta che un altro thread invoca `cond_signal()` o `cond_broadcast()` per svegliare uno o tutti i thread in attesa, il thread risvegliato compete per riottenere il mutex e riprende l'esecuzione. Tuttavia, è importante notare che non è garantito che il thread risvegliato prosegua immediatamente con l'esecuzione. Infatti, a causa della natura della sincronizzazione concorrente, il thread potrebbe svegliarsi solo per scoprire che la condizione per cui era in attesa non è ancora soddisfatta. In questo caso, il thread dovrà richiamare nuovamente `cond_wait()` per riprendere l'attesa fino a quando la condizione desiderata sarà finalmente soddisfatta. Questo fenomeno è noto come "spurious wakeup" e rappresenta una condizione che può verificarsi nei sistemi concorrenti. Pur senza avere l'ambizione di affrontare la discussione del fenomeno nella sua complessità, nella sezione 6.2.2 di queste dispense ne viene introdotta l'origine e le buone prassi per la sua gestione. Per evitare che il thread prosegua erroneamente senza che la condizione sia stata effettivamente soddisfatta, è buona pratica avvolgere il ciclo di attesa e il controllo della condizione in un ciclo `while`, garantendo che il thread si risvegli solo quando la condizione desiderata è effettivamente vera.

6.2.2 Spurious Wakeups

Le *spurious wakeups* sono risvegli inattesi di un thread in attesa su una condition variable, senza che vi sia stata una segnalazione esplicita tramite `cond_signal` o `cond_broadcast`. POSIX specifica esplicitamente che questi risvegli possono avvenire e che i programmi devono essere progettati per gestirli correttamente.

Origine degli Spurious Wakeups

Le spurious wakeups possono verificarsi per diversi motivi, tra cui:

- ▶ Implementazioni specifiche dei thread scheduler, che possono sbloccare erroneamente più thread in attesa su una stessa condition variable.
- ▶ Ottimizzazioni delle prestazioni nei sistemi multiprocessore, dove la gestione delle condition variables può consentire il risveglio di più thread del necessario.
- ▶ Effetti collaterali di meccanismi di sospensione e risveglio gestiti dall'hardware o dal sistema operativo.

Un esempio tratto dalla documentazione POSIX 2024 illustra come, in un sistema multiprocessore, più thread possano essere risvegliati per una singola segnalazione. Consideriamo la seguente implementazione semplificata di `pthread_cond_wait()` e `pthread_cond_signal()`:

```

1 pthread_cond_wait(mutex, cond):
2     value = cond->value;           /* 1 */
3     pthread_mutex_unlock(mutex);   /* 2 */
4     pthread_mutex_lock(cond->mutex); /* 10 */
5     if (value == cond->value) {    /* 11 */
6         me->next_cond = cond->waiter;
7         cond->waiter = me;
8         pthread_mutex_unlock(cond->mutex);
9         unable_to_run(me);
10    } else
11        pthread_mutex_unlock(cond->mutex); /* 12 */
12    pthread_mutex_lock(mutex);      /* 13 */
13
14 pthread_cond_signal(cond):
15     pthread_mutex_lock(cond->mutex); /* 3 */
16     cond->value++;                  /* 4 */
17     if (cond->waiter) {            /* 5 */
18         sleeper = cond->waiter;    /* 6 */
19         cond->waiter = sleeper->next_cond; /* 7 */
20         able_to_run(sleeper);      /* 8 */
21    }
22    pthread_mutex_unlock(cond->mutex); /* 9 */

```

Listato 6.2: implementazione semplificata di `pthread_cond_wait()` e `pthread_cond_signal()`

Qui, un thread può misurare un valore associato alla condition variable (`cond->value`) prima di entrare in attesa. Tuttavia, se un altro thread modifica tale valore mentre il primo è in fase di sospensione, il risveglio può avvenire senza una segnalazione esplicita.

Gestione degli Spurious Wakeups

Per gestire correttamente questi risvegli inattesi, è fondamentale che le attese su una condition variable siano sempre racchiuse in un ciclo di verifica della condizione attesa. In altre parole, i thread non devono assumere di essere stati risvegliati per una ragione valida, ma devono ricontrollare il predicato prima di procedere.

Un'implementazione corretta utilizza un ciclo `while` attorno alla chiamata di attesa:

```
pthread_mutex_lock(&mutex);
while (!condition_met) {
    pthread_cond_wait(&cond, &mutex);
}
pthread_mutex_unlock(&mutex);
```

Questo approccio garantisce che, anche in caso di spurious wakeup, il thread verifichi se la condizione è effettivamente soddisfatta prima di proseguire.

Perché POSIX (ma non solo) permette le Spurious Wakeups?

La documentazione POSIX 2024 * chiarisce che, sebbene sia tecnicamente possibile eliminare le spurious wakeups, ciò comporterebbe una significativa penalizzazione delle prestazioni nei sistemi multiprocessore. Eliminare completamente questo fenomeno implicherebbe un controllo più rigido sui risvegli, con un costo computazionale elevato.

Inoltre, il requisito di verificare sempre la condizione attesa in un ciclo `extttwhile` rende le applicazioni più robuste e resilienti anche a errori accidentali in altre parti del codice che potrebbero causare segnali superflui su una condition variable.

In sintesi, le spurious wakeups sono un effetto collaterale del modello di sincronizzazione POSIX, ma possono essere gestite in modo sicuro e prevedibile utilizzando le giuste tecniche di programmazione concorrente.

6.2.3 Esempio di Produttori-Consumatori con Condition Variables

Il codice riportato nei Listati 6.3 e 6.4 implementa un modello di produttori e consumatori. Nel programma, sono definiti due tipi di thread: produttori e consumatori, che operano su un buffer condiviso, un “distributore di bibite”. I produttori aggiungono bibite al buffer, mentre i consumatori le prelevano. Il buffer ha una capacità massima definita da `K_BUFFER`, ed è protetto da un mutex (`mtx_t mutex`) per garantire che solo un thread alla volta possa accedervi.

Quando il buffer è pieno, i produttori devono attendere che il buffer si svuoti, mentre quando il buffer è vuoto, i consumatori devono aspettare che un produttore aggiunga una bibita. Le variabili di condizione (`cnd_t pieno` e `cnd_t vuoto`) sono utilizzate per sincronizzare i thread: i produttori si bloccano quando il buffer è pieno e i consumatori quando

* <https://pubs.opengroup.org/onlinepubs/9799919799/>

il buffer è vuoto. Le funzioni `cnd_wait` e `cnd_broadcast` vengono usate per far sospendere un thread in attesa di una condizione e svegliare tutti i thread in attesa quando la condizione cambia.

Il programma simula il comportamento dei produttori e consumatori attraverso cicli infiniti, dove ogni produttore e consumatore si sospende per un intervallo di tempo casuale, simulando il tempo di produzione e consumo delle bibite. La sincronizzazione tra i thread avviene grazie al mutex, che impedisce race condition. Una volta terminato l'elaborato, i mutex e le variabili di condizione vengono distrutti.

Il codice gestisce correttamente le condizioni di attesa attraverso l'uso di variabili di condizione (`cnd_t pieno` e `cnd_t vuoto`) e mutex per proteggere l'accesso al buffer. È importante notare che le chiamate alla funzione `cnd_wait` sono inserite all'interno di un ciclo `while` per gestire i cosiddetti spurious wakeups (risvegli spurii).

Come già accennato in precedenza, per gestire correttamente questa situazione, è prassi comune inserire la chiamata a `cnd_wait` all'interno di un ciclo `while`. Il ciclo verifica se la condizione è effettivamente soddisfatta (ad esempio, se il buffer non è più pieno per un produttore o se il buffer non è più vuoto per un consumatore). Se la condizione non è soddisfatta, il thread continua a rimanere in attesa. Se il thread viene risvegliato da un altro thread, il ciclo riesamina la condizione e, se necessario, il thread continuerà ad aspettare.

Nel caso di questo codice, il ciclo `while (buffer == K_BUFFER)` nel produttore e `while (buffer == 0)` nel consumatore assicurano che il thread continui a rimanere in attesa finché il buffer non raggiunge uno stato che consenta l'operazione desiderata (aggiungere una bibita per il produttore o prelevarne una per il consumatore).

Questa gestione è fondamentale per evitare che i thread possano risvegliarsi erroneamente e proseguire l'esecuzione senza aver rispettato la corretta sincronizzazione.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <threads.h>
4  #include <time.h>
5
6  #define M_PRODUTTORI 5           // Numero di produttori
7  #define N_CONSUMATORI 10        // Numero di consumatori
8  #define K_BUFFER 5             // Dimensione del buffer
9  #define MEZZOSECONDO 5.0e8      // Mezzo secondo in nanosecondi
10
11 int buffer = 0;                  // Numero di bibite nel distributore
12 mtx_t mutex;                     // Mutex per la mutua esclusione
13 cnd_t pieno, vuoto;             // Variabili di Condizione
14
15 int produttore(void *arg) {      // Funzione produttore
16     int id = *(int *)arg;        // ID del produttore
17     while (1) {                  // In un ciclo infinito
18         thrd_sleep(              // Simula il tempo di produzione
19             &(struct timespec)
20             {.tv_nsec = MEZZOSECONDO},
21             NULL
22         );
23
24         mtx_lock(&mutex);         // Blocca il mutex
25         while (buffer == K_BUFFER) // Se il buffer risulta pieno
26             cnd_wait(&pieno, &mutex); // Aspetta che il buffer si svuoti
27
28         buffer++;                 // Aggiunge una bibita
29         printf( "Produttore %d " // Stampa azione a video
30             "ha aggiunto una bibita. "
31             "Totale: %d\n", id, buffer
32         );
33
34         cnd_broadcast(&vuoto);    // Sveglia TUTTI i consumatori
35         mtx_unlock(&mutex);        // Sblocca il mutex
36     }
37 }
38
39 int consumatore(void *arg) {     // Funzione consumatore
40     int id = *(int *)arg;        // ID del consumatore
41     while (1) {                  // In un ciclo infinito
42
43         thrd_sleep(              // Simula il tempo di consumazione
44             &(struct timespec)
45             {.tv_sec = 1},
46             NULL
47         );
48
49         mtx_lock(&mutex);         // Blocca il mutex
50         while (buffer == 0)        // Se il buffer risulta vuoto
51             cnd_wait(&vuoto, &mutex); // Aspetta che il buffer si riempia
52
53         buffer--;                 // Sottrae una bibita
54         printf( "Consumatore %d " // Stampa azione a video
55             "ha preso una bibita. "
56             "Totale: %d\n", id, buffer
57         );
58
59         cnd_broadcast(&pieno);    // Sveglia tutti i produttori
60         mtx_unlock(&mutex);        // Sblocca il mutex
61     }
62 }

```

Listato 6.3 Produttore-Consumatore con Condition Variables (funzioni Produttore e Consumatore)

```

1 int main() {
2     srand(time(NULL));
3
4     thrd_t produttori[M_PRODUTTORI];
5     thrd_t consumatori[N_CONSUMATORI];
6     int id_prod[M_PRODUTTORI], id_cons[N_CONSUMATORI];
7
8     mtx_init(&mutex, mtx_plain);
9     cnd_init(&pieno);
10    cnd_init(&vuoto);
11
12    // Creazione produttori
13    for (int i = 0; i < M_PRODUTTORI; i++) {
14        id_prod[i] = i + 1;
15        thrd_create(&produttori[i], produttore, &id_prod[i]);
16    }
17
18    // Creazione consumatori
19    for (int i = 0; i < N_CONSUMATORI; i++) {
20        id_cons[i] = i + 1;
21        thrd_create(&consumatori[i], consumatore, &id_cons[i]);
22    }
23
24    // Attende i thread (in un'app reale, servirebbe una condizione di terminazione)
25    for (int i = 0; i < M_PRODUTTORI; i++) {
26        thrd_join(produttori[i], NULL);
27    }
28    for (int i = 0; i < N_CONSUMATORI; i++) {
29        thrd_join(consumatori[i], NULL);
30    }
31
32    mtx_destroy(&mutex);
33    cnd_destroy(&pieno);
34    cnd_destroy(&vuoto);
35
36    return 0;
37 }

```

Listato 6.4 Produttore-Consumatore con Condition Variables (funzione main)

